



PDF Download
3658644.3670300.pdf
29 December 2025
Total Citations: 3
Total Downloads: 2689



Published: 02 December 2024

Citation in BibTeX format

CCS '24: ACM SIGSAC Conference on
Computer and Communications Security
October 14 - 18, 2024
UT, Salt Lake City, USA

Conference Sponsors:
SIGSAC

DL Latest updates: <https://dl.acm.org/doi/10.1145/3658644.3670300>

RESEARCH-ARTICLE

CoGNN: Towards Secure and Efficient Collaborative Graph Learning

ZHENHUA ZOU, Tsinghua University, Beijing, China

ZHUOTAO LIU, Tsinghua University, Beijing, China

JINYONG SHAN

QI LI, Tsinghua University, Beijing, China

KE XU, Tsinghua University, Beijing, China

MINGWEI XU, Tsinghua University, Beijing, China

Open Access Support provided by:

Tsinghua University



CoGNN: Towards Secure and Efficient Collaborative Graph Learning

Zhenhua Zou
Tsinghua University
Beijing, China
zou-zh21@mails.tsinghua.edu.cn

Zhuotao Liu*
Tsinghua University
& Zhongguancun Laboratory
Beijing, China
zhuotaoliu@tsinghua.edu.cn

Jinyong Shan
Sudo Technology
Beijing, China
jnnngshan@gmail.com

Qi Li
Tsinghua University
& Zhongguancun Laboratory
Beijing, China
qli01@tsinghua.edu.cn

Ke Xu
Tsinghua University
& Zhongguancun Laboratory
Beijing, China
xuke@tsinghua.edu.cn

Mingwei Xu
Tsinghua University
& Zhongguancun Laboratory
Beijing, China
xumw@tsinghua.edu.cn

ABSTRACT

Collaborative graph learning represents a learning paradigm where multiple parties jointly train a graph neural network (GNN) using their own proprietary graph data. To honor the data privacy of all parties, existing solutions for collaborative graph learning are either based on federated learning (FL) or secure machine learning (SML). Although promising in terms of efficiency and scalability due to their distributed training scheme, FL-based approaches fall short in providing provable security guarantees and achieving good model performance. Conversely, SML-based solutions, while offering provable privacy guarantees, are hindered by their high computational and communication overhead, as well as poor scalability as more parties participate.

To address the above problem, we propose CoGNN, a novel framework that simultaneously reaps the benefits of both FL-based and SML-based approaches. At a high level, CoGNN is enabled by (i) a novel message passing mechanism that can obviously and efficiently express the vertex data propagation/aggregation required in GNN training and inference and (ii) a two-stage Dispatch-Collect execution scheme to securely decompose and distribute the GNN computation workload for concurrent and scalable executions. We further instantiate the CoGNN framework, together with customized optimizations, to train Graph Convolutional Network (GCN) models. Extensive evaluations on three graph datasets demonstrate that compared with the state-of-the-art (SOTA) SML-based approach, CoGNN reduces up to 123x running time and up to 522x communication cost per party. Meanwhile, the GCN models trained using CoGNN have nearly identical accuracies as the plaintext global-graph training, yielding up to 11.06% accuracy improvement over the GCN models trained via federated learning.

*Zhuotao Liu is the corresponding author.



This work is licensed under a Creative Commons Attribution International 4.0 License.

CCS '24, October 14–18, 2024, Salt Lake City, UT, USA
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0636-3/24/10
<https://doi.org/10.1145/3658644.3670300>

CCS CONCEPTS

• Security and privacy → Cryptography; • Computing methodologies → Machine learning.

KEYWORDS

Collaborative Graph Learning, Secure Multi-party Computation

ACM Reference Format:

Zhenhua Zou, Zhuotao Liu, Jinyong Shan, Qi Li, Ke Xu, and Mingwei Xu. 2024. CoGNN: Towards Secure and Efficient Collaborative Graph Learning. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS '24)*, October 14–18, 2024, Salt Lake City, UT, USA. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3658644.3670300>

1 INTRODUCTION

Recent years have witnessed the success of deploying Graph Neural Network (GNN) [19, 25, 47] in areas like financial fraud detection [50, 52] and drug discovery [22, 57]. As a special category of neural network, GNN takes as input a graph, composed of vertices and the edges connecting vertices. Commonly, the goal of a GNN is to classify vertices, which is called vertex-level (or node-level) prediction. Other tasks include classifying edges or the whole graph. In vertex-level prediction, each vertex is a sample with its features, while each edge connects two vertices and is assigned a weight. The edge weight implicates how the two vertices are related to each other. The key feature distinguishing GNN from the other machine learning models is that it considers the edges (i.e., relations) between different samples during forward and backward computation, while the other models predict on each sample independently. Specifically, the computation of GNN consists of two stages. First, it propagates the features or hidden-layer representation of each sample to influence its neighbors, via the graph edges. This process is called *message passing* [15, 48, 55]. Second, it performs NN computation on the result from the previous stage, as in a traditional NN model.

In this paper, we study the collaborative learning and inference of GNN, where multiple graph data owners jointly train a global GNN model by contributing their *private* graph data. These graph owners have exclusively different sample sets (i.e., vertices), but there could be *inter-edges* connecting the vertices (or graphs) held

by different owners. For example, different banks have different sets of accounts, but their transaction graphs are interleaved into a global transaction graph across banks by inter-bank transactions. Interleaved communication graphs of multiple mobile carriers are similar. Jointly analyzing the collection of inter-related graphs is critical to a number of use cases. For instance, to detect fraud accounts across multiple inter-related financial systems [46] or mobile carriers [21], collaboratively training GNNs using data from multiple organizations results in supreme model performances.

However, as data privacy becomes increasingly important (either due to commercial interests or legal compliance (e.g., GDPR [1]), honoring data privacy in collaborative GNN is challenging. The community has proposed significant research in this regard. The first category is based on Federated Learning (FL) [28]. In particular, these FL-based approaches [42, 51, 53, 56] have the participants locally execute graph message passing and NN computation, and coordinate them by performing gradient/model weight aggregation. The FL-based approaches are efficient because all the parties compute their local graph concurrently. They do not support message passing across the inter-edges because passing vertex embeddings/hidden-layer representations on inter-edges in plaintext raises privacy concerns. However, ignoring inter-edge messaging passing results in non-trivial performance degradation of the GNN model, as shown in [51]. In addition, the intermediate data disclosed during federated training (such as the global gradient) raises further privacy concerns, which has been exploited by existing work [35, 40]. Some recent FL-based approaches [13, 31] enhance their privacy (security) guarantees by incorporating cryptographic techniques like secure multi-party computation (MPC) [11, 36] and homomorphic encryption (HE) [23], where they focus on protecting the model/gradient aggregation process. However, they fail to provide an end-to-end provable privacy guarantee, because the aggregated intermediate results (like the global gradients) are either revealed in plaintext to the clients [31] (introducing privacy vulnerabilities as stated above) or hidden via TEE hardware [13] (vulnerable to various attacks [24, 39]).

On the other hand, another branch of prior approaches leverages secure machine learning (SML) to provide end-to-end security guarantees. These approaches [26, 49] work in an outsourced setting, where all graph owners outsource their data, in the secret-shared form, to several third-party computing servers. The servers jointly train the GNN model and only send back the trained model, without disclosing any intermediate results. The data privacy is provably guaranteed via the underlying MPC protocols [11, 27, 36]. However, these approaches are limited by two drawbacks: (i) *poor efficiency*: the computation/communication cost in the outsourced setting is high, mainly caused by expensive cryptographic operations like oblivious sort [5, 26] or secure array accesses [8]. For instance, experimental results of [5] show that oblivious sort takes up more than 70% of the overall running time of graph algorithms; (ii) *poor scalability*: unlike the distributed computation in FL where each party processes its local graph only, each party in SML-based solutions has to process the entire global graph, resulting in poor scalability as the number of graph owners increases (i.e., as the scale of the global graph increases). Therefore, scaling SML-based approaches to compute large global graphs is challenging.

In summary, in supporting collaborative GNN training and inference, existing solutions face critical limitations. FL-based approaches, while offering distributed computation, lack provable privacy guarantees and suffer from limited model performance. Conversely, SML-based have desirable security properties, but at the cost of significant overhead and limited scalability. In this paper, we seek to answer the following research question: *can we combine the benefits of both categories of approaches: achieving provable privacy guarantees and high model performance (in line with the SML-based approaches), while adopting a fully distributed computation scheme (in line with the FL-based approaches) to enable supreme efficiency and scalability.*

Towards this end, we propose CoGNN, a novel framework to enable secure and efficient collaborative GNN learning. CoGNN is empowered by two fundamental designs: (i) a novel *oblivious message passing paradigm* to support the vertex data propagation/aggregation in GNN training and inference, which reduces the communication cost by half compared with the state-of-the-art (SOTA) (see § 5.1); (ii) a *two-stage Dispatch-Collect* execution scheme built upon the message passing paradigm to securely and efficiently decompose and distribute the workload of GNN computation across all parties for scalable computation (see § 5.2). In particular, our oblivious message passing paradigm stores vertices and edges as separated lists and uses efficient and oblivious interaction between the two lists to express vertex data propagation/aggregation. At the same time, the two-stage Dispatch-Collect decomposes GNN computation (including message passing) into Dispatch tasks and Collect tasks and distributes these tasks to different parties for concurrent and scalable execution. Each task is a secure computation workload executed by a pair of parties. The task distribution or assignment leverages the secret topology information held by each party to avoid computationally expensive cryptographic operations (e.g., oblivious sort or secure array access).

We instantiate the CoGNN framework to train Graph Convolutional Network (GCN) models and provide detailed construction of all the Dispatch and Collect tasks to enable GCN computations (see § 6). The instantiation includes further optimizations to improve the concrete efficiency of computing GCN (see § 6.6).

Our implementation of CoGNN includes ~ 6800 lines of C++ code. We evaluate it extensively on commonly used graph datasets (see § 8). Compared to the state-of-the-art SML-based approach in training GCN, CoGNN reduces the running duration and per-party communication significantly. The advantages are enlarged as the number of graph owners increases, indicating better scalability of CoGNN. In particular, when there are 5 graph owners, CoGNN reduces the running duration by 123x, and reduces per-party communication by 522x. Moreover, CoGNN has a model performance comparable to plaintext global-graph training and surpasses the FL-based approach significantly. Compared to the FL-based approach, CoGNN has an accuracy elevation up to 3.87% in the two-party setting, and up to 11.06% in the five-party setting.

Contributions. The primary contributions of this paper are the design, implementation and evaluation of CoGNN, a novel framework to enable collaborative GNN learning in a provably secure, efficient and scalable manner. Concretely, our contributions are summarized as follows:

- We propose CoGNN, a secure, efficient and scalable framework for collaborative GNN learning among multiple graph owners. Inside CoGNN, we design a novel oblivious message-passing mechanism that halves the communication required by the SOTA approach. Based on this, we introduce a two-stage Dispatch-Collect scheme to securely and efficiently execute the workload of GNN training and inference.
- We instantiate the CoGNN framework with concrete protocols to train Graph Convolutional Networks (GCNs). Meanwhile, we propose customized optimizations tailored for GCNs to further improve its learning efficiency.
- We implement the CoGNN framework and evaluate it extensively on multiple datasets used in GCN learning. Experimental results show that, compared to the SOTA SML-based approach, CoGNN reduces up to 123x running time and up to 522x communication cost per party. The improvement is enlarged as the number of parties increases, demonstrating the superior scalability of CoGNN. Meanwhile, the GCN models trained using CoGNN have nearly identical accuracies as the plaintext global-graph training, yielding up to 11.06% accuracy improvement over the GCN models trained via federated learning.

2 BACKGROUND & PREVIOUS WORKS

2.1 Graph

We study a directed graph $G = (V, E)$, where V (E) is the vertex (edge) list. Each vertex $v \in V$ is a tuple $(v.id, v.data)$, where $v.id$ is a unique vertex identifier and $v.data$ is the vertex data. Each edge $e \in E$ is $(e.src, e.dst, e.data)$, where $e.src$ ($e.dst$) is the identifier of the source (destination) vertex. We call edge e the outgoing edge of $e.src$ and the incoming edge of $e.dst$. The number of outgoing (incoming) edges of a vertex is called its outgoing (incoming) degree. An undirected graph can be converted to an equivalent directed graph by decomposing each undirected edge into two directed edges in opposite directions. For simplicity, we use v_i to denote v that $v.id = i$, and use e_{ij} to denote e that $e.src = i, e.dst = j$.

2.2 Graph Neural Network

The input of GNN is a directed graph $G = (V, E)$, where V is the sample list and E connects the samples. Specifically, $v.data$ contains the vertex features, while $e.data$ stores the edge weight. We introduce the computation of GNN from a general perspective, leveraging the vertex-centric (or think-like-a-vertex) abstraction [32, 33]. In particular, a forward/backward GNN layer can be abstracted as the following three consecutive operations (i.e., the GAS model):

Scatter. For $\forall v \in V$, propagate the vertex data to each of its outgoing edges e and outputs an *update* u :

$$u \leftarrow \text{Scatter}(v.data, e.data), e.src = v.id$$

Gather. For $\forall v \in V$, aggregate the update data u originating from all of its incoming edges e with $v.data$:

$$v.data \boxplus \{u\} \leftarrow \text{Gather}(v.data, \{u \mid u.dst = v.id\})$$

Apply. Perform NN computation (like weight multiplication, non-linear activation) on $v.data \boxplus \{u\}$ to obtain the updated $v.data'$:

$$v.data' \leftarrow \text{Apply}(v.data \boxplus \{u\})$$

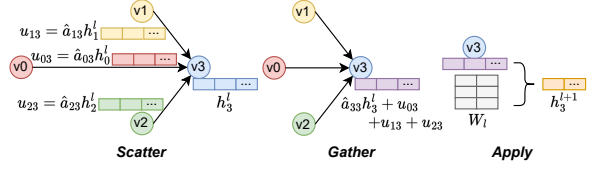


Figure 1: GCN forward pass as vertex-centric computation.

The Scatter-Gather operations are called *message passing*. Different GNN models vary in their constructions of the three operations.

2.3 Graph Convolutional Network

We elaborate on the most used GNN model, i.e., Graph Convolutional Network (GCN), and how it corresponds to the GAS model.

Forward Pass. The computation of GCN centers around the adjacency matrix $A \in \{0, 1\}^{|V| \times |V|}$, which expresses E ($a_{ij} = 1$ if and only if $e_{ij} \in E$). The features in V are expressed as a feature matrix X . The computation of the l -th forward layer of GCN is:

$$H_{l+1} = \sigma(\hat{A}H_l W_l). \quad (1)$$

H_l is the l -th layer hidden representation of all the vertices, where $H_0 = X$. W_l is the trainable weight matrix of the l -th layer. $\hat{A}$ is the normalized version of A , where $\hat{A} = \tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}}$, $\tilde{A} = A + I$. $\tilde{D}$ is a diagonal matrix such that $\tilde{d}_{ii} = \sum_j \tilde{a}_{ij}$ (i.e., each diagonal cell storing the degree of a vertex). $\hat{A}H_l$ actually corresponds to the message passing process. $\sigma(\cdot)$ denotes a non-linear activation function, e.g., *ReLU* and *Softmax*. *Softmax* is typically used at the last layer to produce the probabilities of each class for all vertices.

Forward Pass as GAS. In the GAS model, the execution of $\hat{A}H_l$ is decomposed into two steps, which correspond to the Scatter and Gather phases respectively. Figure 1 is an example showing the related GAS computations for v_3 at the forward layer l . At the beginning of this GAS iteration, for each vertex v_i , $v_i.data$ stores its hidden representation h_i^l , which corresponds to a row in H_l . The $e_{ij}.data$ for edge e_{ij} is its weight $\hat{a}_{ij}$, which is an element in $\hat{A}$.

During Scatter, we multiply the hidden representation h_i^l of each vertex v_i with the weight $\hat{a}_{ij}$ on its corresponding outgoing edge to create the update $u_{ij} = \hat{a}_{ij}h_i^l$ for its destination vertex v_j . During Gather, we add up the hidden representation h_j^l of the destination vertex v_j with all of its incoming updates. The merge (aggregate) operation, i.e., $\boxplus$, of GCN is a weighted sum. For example, in Figure 1, $h_3^l \boxplus u_{03} \boxplus u_{13} \boxplus u_{23} = \hat{a}_{33}h_3^l + u_{03} + u_{13} + u_{23}$.

During Apply, the result of each vertex from Gather is first multiplied with the weight matrix W_l , and then passed to the activation function σ . The final output of Apply for v_j is h_j^{l+1} , i.e., the hidden representation of the next layer. By combining all h_j^{l+1} from each v_i , we obtain H_{l+1} in Equation (1).

Backward Pass. The backward pass of a GCN produces gradients for all the trainable weights following the chain rule. The backward computation of GCN at the $(l-1)$ -th layer is defined as:

$$D_{l-1} = (\hat{A}H_{l-1})^T (\sigma'(Z_{l-1}) \odot \hat{A}^T G_l), \quad (2)$$

$$G_{l-1} = (\sigma'(Z_{l-1}) \odot \hat{A}^T G_l) W_{l-1}^T. \quad (3)$$

D_l denotes the gradient w.r.t. W_l , while G_l denotes the gradient w.r.t. $\hat{A}H_l$. σ' is the derivation of σ . $\odot$ represents elementwise multiplication, while $(\cdot)^T$ represents the transposition of a matrix. $Z_l =$

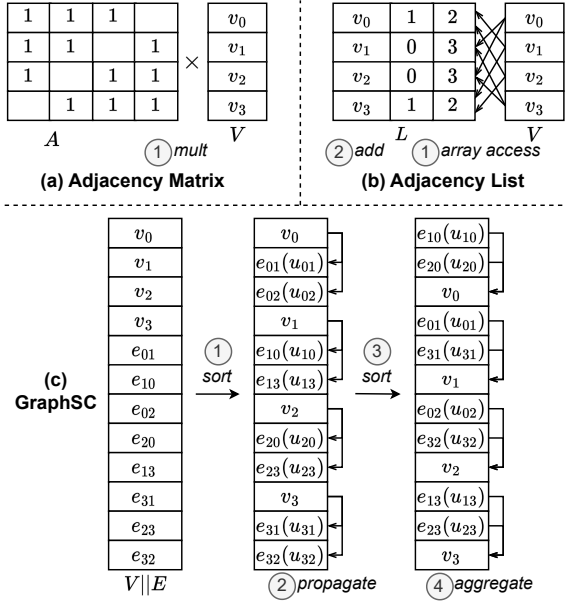


Figure 2: Prior approaches of oblivious message passing.

$\hat{A}H_lW_l$ is computed during the forward pass. When using cross-entropy loss, for a K -layer GCN, $D_{K-1} = (\hat{A}H_{K-1})^T(\hat{Y} - Y)$, where Y is the ground-truth prediction results. $G_{K-1} = (\hat{Y} - Y)W_{K-1}^T$.

Backward Pass as GAS. From Equation (3) we can see that the computation of G_l is symmetric to H_l . Specifically, to derive G_{l-1} from G_l , we first multiply it with $\hat{A}^T$, and then perform element-wise multiplication with $\sigma'(Z_{l-1})$ before multiplying W_{l-1}^T . As a result, when expressing GCN backward layer $l-1$ in the GAS model, the input data of each vertex v_j is its corresponding row in G_l , i.e., g_j^l . The Scatter and Gather phases execute $\hat{A}^T$, while the Apply phase performs the multiplication with $\sigma'(Z_{l-1})$ and W_{l-1}^T . The output of Apply for v_j is its corresponding row in G_{l-1} , i.e., g_j^{l-1} .

The byproduct of Apply in each vertex v_j is a gradient matrix D_{l-1}^j for W_{l-1} . It is derived by multiplying v_j 's corresponding column of $(\hat{A}H_{l-1})^T$ with its row in $\sigma'(Z_{l-1}) \odot \hat{A}^T G_l$. Note that, D_{l-1} is actually the sum of the gradient matrices D_{l-1}^j generated in all vertices v_j . To obtain D_{l-1} , at the end of the corresponding backward GAS iteration, we add an aggregation step to average the gradients produced in each vertex. D_{l-1} can then be applied to update the weight matrix W_{l-1} .

2.4 Previous Works

The main technical challenge in achieving provably secure GNN learning in SML-based approaches is to efficiently perform oblivious message passing. In this section, we overview and compare prior approaches of oblivious message passing. Without loss of generality, we discuss a simplified case of message passing, i.e., the Scatter operation copies v .data to all $\{u \mid u.\text{src} = v.\text{id}\}$ and the Gather operation adds up v .data and all $\{u \mid u.\text{dst} = v.\text{id}\}$. In the form of adjacency matrix, it equals computing AH , where $A \in \{0, 1\}^{|V| \times |V|}$. Suppose that A is held by P_0 , while H is additively secret-shared between P_0 and P_1 . The size of each element of H is m bits. For fair comparison, we assume that all costly operations,

e.g., multiplication, array access and sort, have been preprocessed offline and focus on comparing the optimal online efficiency.

Adjacency Matrix-Based. The naive approach is to execute AH directly as secure matrix multiplication, as shown in (a) of Figure 2. H corresponds to the vertex list V . Utilizing the matrix multiplication triple [38] prepared offline, the online communication overhead (send + receive) of each party is $2m|V|^2 + 2m|V|$. However, [38] shows that the offline duration is over 10 times the online duration.

Adjacency List-Based. For a sparse adjacency matrix A , SecGNN [49] stores A as an adjacency list L and avoids the huge effort of pre-processing matrix multiplication triples by retrieving vertex data from the vertex list using secure array access [8]. As shown in (b) of Figure 2, each row of L records a vertex and the indices of all the vertices connected to it. SecGNN appends dummy indices to each row to make the degree of each vertex match the highest degree in the original graph. The message passing process specified by AH is then decomposed into two steps: (1) for each neighbor of each vertex, run an oblivious array access operation [8] to V to extract the corresponding vertex data; (2) for each vertex, sum up its neighbors' data obtained from the previous step. Since the online communication of each array access is $m|V|$, the method of [49] has a total online communication of at least $m|E||V|$, i.e., when no dummy indices are added.

GraphSC-Based. Recently, Entrada [26] introduced GraphSC [41], a vertex-centric secure graph processing paradigm, to secure GNN computation. As shown in (c) of Figure 2, the GraphSC paradigm works by storing vertices and edges in the same list $V||E$, where E expresses the graph topology defined in A . Executing AH takes four steps: (1) sort $V||E$ to place the edges right after their source vertices; (2) obviously propagate the data of each vertex to its outgoing edges to generate updates; (3) sort $V||E$ again to place the edges right before their destination vertices; (4) obviously aggregate the updates of each edge to their destination vertices. The sort operations in steps (1) and (3) can be specified by a permutation π and executed using permutation correlations online [12]. Each sort takes $m(|E| + |V|)$ online communication. The original propagate and aggregate schemes provided in [41] are both $O(n \log(n))$, where $n = |E| + |V|$. Recent work [7] shows that they can be replaced by prefix adder network-based oblivious computation to achieve linear complexity. In our case, each of propagate and aggregate takes $2(|E| + |V|)$ OTs and requires at least $4m(|E| + |V|)$ online communication. Taken together, the overall communication cost of executing AH using the GraphSC paradigm is at least $10m(|E| + |V|)$. GraphSC surpasses the two prior approaches significantly and is the current state-of-the-art approach for secure GNN computation, mainly due to its linear complexity w.r.t. $|E| + |V|$.

3 PRELIMINARIES

3.1 Cryptographic Primitives

Fixed-Point Encoding and Secret Share. In CoGNN, we encode all the graph data as a Fixed-Point representation over the ring, $\mathbb{Z}_L$, where $L := 2^l$. The 2-out-of-2 additive secret share of a fixed-point number x over $\mathbb{Z}_L$ is straightforward. We denote the i -th share as $\langle x \rangle_i$, $i \in \{0, 1\}$. By randomly sampling $\langle x \rangle_0 \in \mathbb{Z}_L$, we get $\langle x \rangle_1 = x - \langle x \rangle_0 \pmod{L}$. We use $\langle x \rangle$ to denote that x is in secret-shared form. For simplicity, we omit the $\langle \cdot \rangle$ symbol in most of our

$\mathcal{F}_{\text{CoGNN}}: (\{E_i\}, \{E_{i,j}\}, \{V_i\}, \{W_l\}, \text{alg}) \rightarrow (\{V'_i\}, \{W'_l\})$ $\mathcal{F}_{\text{CoGNN}}$ interacts with N parties, $(P_0, P_1, \dots, P_{N-1})$. Input: (1) For $i \in [N]$, P_i sends $(E_i, E_{i,j}, V_i)$ to $\mathcal{F}_{\text{CoGNN}}$; (2) All P_i agree on a GNN specification, alg , which defines the computation details, e.g., GNN model, training epoch, loss function, and learning rate. They send alg to $\mathcal{F}_{\text{CoGNN}}$. (3) All P_i agree on the initial model weight of each layers, i.e., $\{W_l\}, l \in [K]$. They send $\{W_l\}$ to $\mathcal{F}_{\text{CoGNN}}$. Compute: (1) $\mathcal{F}_{\text{CoGNN}}$ concatenates the received subgraph data as a global graph G , where $G = (V, E)$, $V := \cup V_i$ and $E := (\cup E_i) \cup (\cup E_{i,j})$; (2) $\mathcal{F}_{\text{CoGNN}}$ runs GNN training/inference on G according to the specification of alg . It stores the obtained vertex embedding in the updated vertex lists $\{V'_i\}, i \in [N]$ and stores the trained model weights in $\{W'_l\}, l \in [K]$. Output: (1) For $i \in [N]$, $\mathcal{F}_{\text{CoGNN}}$ sends V'_i and $\{W'_l\}, l \in [K]$ to P_i ; (2) Additionally, $\mathcal{F}_{\text{CoGNN}}$ sends the subgraph sizes to P_i , i.e., $\mathcal{L} := \{ E_j , V_j , \forall j \in [N] \setminus \{i\}\}$ representing the numbers of intra-edges and vertices of each subgraph.

Functionality 1: $\mathcal{F}_{\text{CoGNN}}$

texts, while including it in the formal specifications of protocols, ideal functionality, and theorems/lemmas.

Secure Computation on Secret-Shared Data. We consider a general-purpose semi-honest two-party secure computation (2PC) protocol over 2-out-of-2 additively secret-shared data, e.g., [14, 43]. Apart from arithmetic operations, e.g., addition, subtraction, multiplication and division, we require logic operations, including comparison and two-way multiplexer (MUX₂) [43].

3.2 Oblivious Vector Operations

In this segment, we introduce the two cryptographic primitives for oblivious vector operations used in our oblivious message passing discussed in § 5.1: *oblivious extended permutation* (OEP) [37] and *oblivious group aggregation* (OGA) [7].

$\mathcal{F}_{\text{OEP}}$ performs an extended permutation π to T_{in} :

$$\langle T_{\text{out}} \rangle = \langle \pi(T_{\text{in}}) \rangle \leftarrow \mathcal{F}_{\text{OEP}}(\langle T_{\text{in}} \rangle, Id_{\text{in}}, Id_{\text{out}}) \quad (4)$$

π is defined by a pair of identifier lists $(Id_{\text{in}}, Id_{\text{out}})$, corresponding to T_{in} and T_{out} respectively. Elements in Id_{in} are unique, while each element of Id_{out} equals one element of Id_{in} . π permutes Id_{in} to Id_{out} . Each vector element in T_{out} (i.e., $T_{\text{out}}[x]$) is a random reshare of a specific vector element in T_{in} (i.e., $T_{\text{in}}[\pi(x)]$). We extend the definition of $\mathcal{F}_{\text{OEP}}$ to support that some identifiers in Id_{out} might not have an equal identifier in Id_{in} , and $\mathcal{F}_{\text{OEP}}$ sets the corresponding elements in $\langle T_{\text{out}} \rangle$ to 0.

$\mathcal{F}_{\text{OGA}}$ aggregates the elements in T_{in} using the operation $\boxplus$:

$$\langle T_{\text{out}} \rangle \leftarrow \mathcal{F}_{\text{OGA}}(\langle T_{\text{in}} \rangle, \mathcal{G}, \boxplus) \quad (5)$$

Specifically, $\mathcal{G}$ is a group identifier vector of the same length as T_{in} , and can be split into segments, each containing elements of the same value. $\mathcal{F}_{\text{OGA}}$ aggregates T_{in} elements with the same group identifier (i.e., in the same segment) and stores the result in the first element of the corresponding segment, producing T_{out} . For the rest of elements in each segment of T_{out} , their values are undefined.

4 THE IDEAL FUNCTIONALITY OF COGNN

In this section, we introduce the ideal functionality of CoGNN (i.e., $\mathcal{F}_{\text{CoGNN}}$, shown in Functionality 1) to formally capture its input setting, computation goal, threat model, and privacy guarantee.

There are N participating parties, i.e., $(P_0, P_1, \dots, P_{N-1})$. P_i locally holds the subgraph $G_i = (V_i, E_i)$. The sets of vertex identifiers held by different parties are exclusively different. Each edge $e \in E_i$ is called an intra-edge since both $e.\text{src}$ and $e.\text{dst}$ are in V_i .

There could be inter-edges between two subgraphs G_i and G_j , $i \neq j$. We use $E_{i,j}$ to denote the edge list directed from G_i to G_j . For $e \in E_{i,j}$, both P_i and P_j learn $(e.\text{src}, e.\text{dst}, e.\text{data})$. Nevertheless, we suppose that only P_i , the source party of all the edges in $E_{i,j}$, needs to maintain it. CoGNN requires that the edge lists $(E_i, E_{i,j}, j \in [N] \setminus \{i\})$ held by each party $P_i, i \in [N]$ are ordered by placing the edges with the same destination vertex as a contiguous *segment* in E_i ($E_{i,j}$) (i.e., these edges are located at adjacent positions).

At the input stage, for $i \in [N]$, P_i sends $(E_i, E_{i,j}, V_i)$ to $\mathcal{F}_{\text{CoGNN}}$. Additionally, they agree on a GNN specification alg and the initial model weights $\{W_0, W_1, \dots, W_{K-1}\}$, and send them to $\mathcal{F}_{\text{CoGNN}}$. In particular, alg specifies GNN computation details such as the GNN model, training epochs, loss function, and learning rate.

During computation, $\mathcal{F}_{\text{CoGNN}}$ first concatenates the received separate subgraphs into a global graph G . The concatenation is based on inter-edges: i.e., $G = (V, E)$, $V := \cup V_i$, $E := (\cup E_i) \cup (\cup E_{i,j})$. Then, $\mathcal{F}_{\text{CoGNN}}$ performs GNN training/inference according to the specification of alg . The computation result consists of two parts: (i) the updated vertex set lists, i.e., $\{V'_0, V'_1, \dots, V'_{N-1}\}$, which store the embedding (prediction values) of each vertex; (ii) the updated model weights, i.e., $\{W'_0, W'_1, \dots, W'_{K-1}\}$, trained on the global graph.

At the output stage, for all $i \in [N]$, $\mathcal{F}_{\text{CoGNN}}$ sends the vertex embedding V'_i and trained model weights $\{W'_0, W'_1, \dots, W'_{K-1}\}$ to P_i . Additionally, $\mathcal{F}_{\text{CoGNN}}$ sends the subgraph size, i.e., numbers of vertices and edges, of each party to P_i .

Threat Model. In CoGNN, we assume that each party is semi-honest, i.e., these parties are protocol compliant, while being curious of deriving other parties' private data. In addition, we assume that these parties do not collude with each other. The rationale behind this threat model is that CoGNN primarily considers the non-outsourced computation setting, where the graph owners themselves act as the computing parties while keeping their raw data local. Therefore, colluding would cause privacy risks to their own data, while behaving maliciously may eventually degrade the overall collaboration gain. In contrast, prior SML-based approaches require the data owners to outsource their raw data to third-party computing servers. Therefore, it is essential for these approaches to ensure security when these third parties collude with each other or even act arbitrarily. In § 9, we discuss how to extend CoGNN to other security models.

Private Data. The private data of party $P_i, \forall i \in [N]$ includes: (i) intra-edge-related information, i.e., $(e.\text{src}, e.\text{dst}, e.\text{data})$ of each edge $e \in E_i$; (ii) $(v.\text{id}, v.\text{data})$ for $v \in V_i$. In addition, all the intermediate data computed using the aforementioned private data from the other subgraphs must be kept secret throughout the process.

Public Data. The public data *between each pair of parties* (P_i and $P_j, i, j \in [N], i \neq j$) includes: (i) inter-edge-related information,

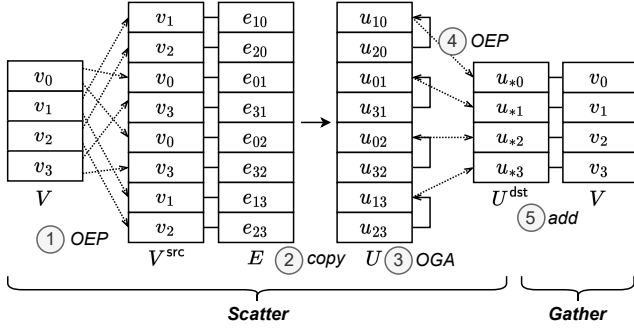


Figure 3: Our oblivious message passing.

i.e., $(e.src, e.dst, e.data)$ of each edge $e \in E_{i,j} \cup E_{j,i}$; (ii) the sizes of subgraphs (i.e., $|V_i|, |V_j|, |E_i|, |E_j|$).

5 THE COGNN FRAMEWORK

In this section, we present the CoGNN framework to securely realizing $\mathcal{F}_{\text{CoGNN}}$ with a fully distributed and secure computation scheme. Specifically, we begin with a novel oblivious message passing mechanism that securely and efficiently expresses the Scatter-Gather of GNN. Based on that, we present how CoGNN securely decomposes and distributes the vertex-centric computation of GNN models across all parties by executing each GNN forward/backward layer in the two stages of Dispatch-Collect.

5.1 Our Oblivious Message Passing

We introduce the principle of our oblivious message passing (Scatter-Gather) method using the same setting as in § 2.4, i.e., when the Scatter operation copies $v.data$ to all $\{u \mid u.src = v.id\}$ and the Gather operation adds up $v.data$ and all $\{u \mid u.dst = v.id\}$.

Instead of storing all the vertices and edges in one list like in GraphSC, we place them in V and E respectively, as shown in Figure 3. Based on that, our oblivious message passing scheme runs in five steps: (1) using OEP (our protocol that realizes the primitive $\mathcal{F}_{\text{OEP}}$) to duplicate each vertex to all of its outgoing edges to create a source vertex list V^{src} ; (2) copy V^{src} to obtain the updates U ; (3) aggregate updates that share the same destination vertices using OGA (our protocol that realizes the primitive $\mathcal{F}_{\text{OGA}}$); (4) Use OEP to place each update at the same vector position as its destination vertex, creating U^{dst} ; (5) locally add updates to vertices.

Steps (1) to (4) correspond to Scatter, while step (5) expresses Gather. Among them, steps (2) and (5) are local. For steps (1) and (4), our OEP protocol works in an offline-online paradigm, where we prepare permutation correlations in batch offline (for multiple online calls), and consume these correlations online. The online communication of each invocation of our OEP protocol is the same as the size of its input vector. As a result, steps (1) and (4) take online $m(|E| + |V|)$ communication in total. Our OGA protocol takes the prefix adder network paradigm, resulting in $4m|E|$ communication. Taken together, our message passing approach has $m(5|E| + |V|)$ online communication, which is less than half of that of the state-of-the-art approach based on GraphSC. More importantly, it enables the following distributed and secure computation scheme in CoGNN. Due to the page limit, the detailed protocol

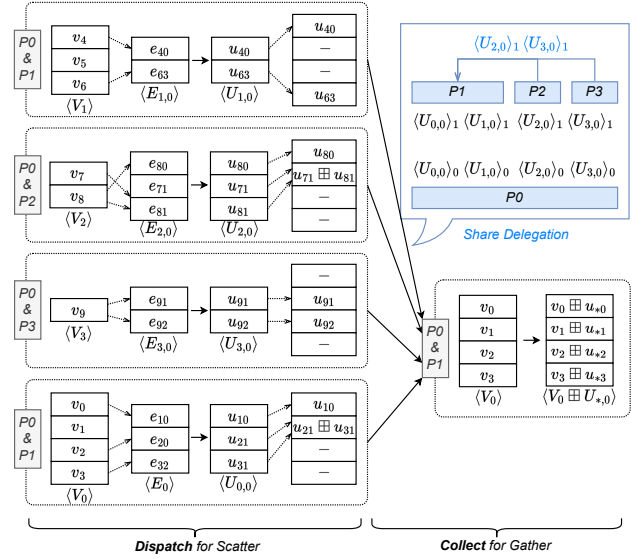


Figure 4: The first-stage Dispatch-Collect in CoGNN: for Scatter and Gather

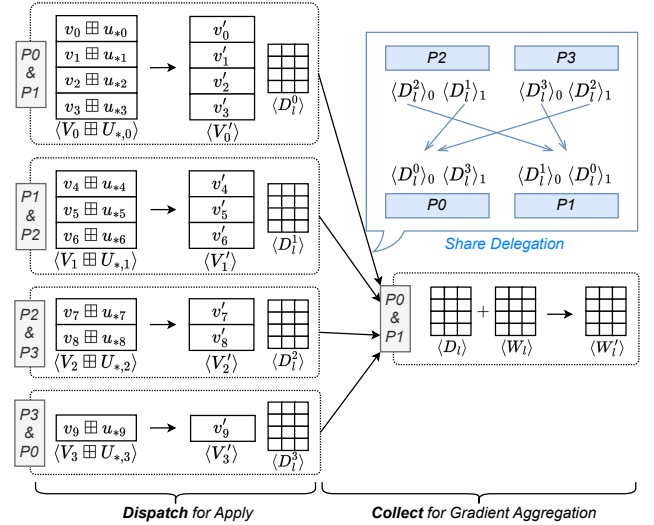


Figure 5: The second-stage Dispatch-Collect in CoGNN: for Apply and Gradient Aggregation

constructions of OEP and OGA are deferred to our technical report [58].

5.2 Iteration as Two-Stage Dispatch-Collect

CoGNN executes each forward/backward layer (i.e., a GAS iteration) of a GNN model using two stages of Dispatch-Collect. The high-level principle of the Dispatch-Collect-based CoGNN design can be summarized as: (i) using the Dispatch phase to securely distribute the heavy part of vertex-centric GNN computation workload across all the parties; (ii) using the Collect phase to perform lightweight aggregation across different parties when necessary. In particular, each Dispatch task involves two worker parties and the computed data is secret-shared between the two parties. During

Collect, the output shares produced by each Dispatch task are delegated to specific aggregator parties, which securely aggregate the data and feed the result shares back to the Dispatch worker parties.

We use a four-party example presented in Figure 4 and Figure 5 as the illustrative example when describing our protocol below.

5.2.1 The First-Stage Dispatch-Collect. In the first-stage Dispatch-Collect, the Dispatch phase processes the Scatter workload in a fully distributed manner, while the Collect phase performs Gather. CoGNN distributes the Scatter workload based on the data partition inherent to its input setting. Specifically, the Scatter from V_j to $E_{j,i}$, $j \in [N] \setminus \{i\}$ is encapsulated as a Dispatch task and is handled by (P_j, P_i) using 2PC. Accordingly, V_j , $E_{j,i}$ and the update $U_{j,i}$ produced by Scatter have to be secret-shared between (P_j, P_i) . As for the Scatter from V_i to E_i , we have (P_i, P_{i+1}) secret-share $(V_i, E_i, U_{i,i})$ and execute the corresponding Scatter computation as a Dispatch task. When $i = N - 1$, $i + 1$ wraps around to 0.

The example of Figure 4 illustrates the Scatter-Gather computations related to the update of V_0 . In Figure 4, the updates targeting V_0 come from four sources, i.e., V_1 , V_2 , V_3 , V_0 . To compute these updates, the Scatter workload is decomposed into four Dispatch tasks, handled by (P_0, P_1) , (P_1, P_2) , (P_2, P_3) , (P_0, P_1) respectively.

The Collect phase, conducting Gather, aims to merge the secret-shared updates $U_{j,i}$, $\forall j \in [N]$ produced by different Dispatch tasks with V_i . Since different $U_{j,i}$ is secret-shared between a different pair of parties (P_j, P_i) , at the beginning of the Collect phase, we have P_j , $j \in [N] \setminus \{i, i + 1\}$ delegate its share of $U_{j,i}$ to P_{i+1} . As a result, (P_i, P_{i+1}) serve as the delegator parties handling the Collect task, i.e., performing the Gather computation for V_i on behalf of the rest of the parties. Nevertheless, all the delegated data is still secret-shared between (P_i, P_{i+1}) and neither of them obtains the plaintext data. The output produced by the Collect task, i.e., $V_i \boxplus U_{*,i}$ representing V_i merged with all the updates from all P_j , is also secret-shared between (P_i, P_{i+1}) . In the example of Figure 4, $\langle U_{1,0} \rangle$, $\langle U_{2,0} \rangle$, $\langle U_{3,0} \rangle$, $\langle U_{0,0} \rangle$ are secret-shared among P_0 and different parties. To apply them to update $\langle V_0 \rangle$, we have P_2, P_3 delegate their shares to P_1 . After that, P_0 and P_1 merge $\langle U_{1,0} \rangle$, $\langle U_{2,0} \rangle$, $\langle U_{3,0} \rangle$, $\langle U_{0,0} \rangle$ with $\langle V_0 \rangle$ consecutively, and obtain the merged $\langle V_0 \boxplus U_{*,0} \rangle$.

5.2.2 The Second-Stage Dispatch-Collect. In the second-stage Dispatch-Collect, the Dispatch phase distributedly processes the Apply workload to perform NN computation on the vertex data of each subgraph and (optionally) produces gradients, while the Collect phase aggregates gradients generated by different subgraphs.

The secret-shared results produced by the Collect phase of the first-stage Dispatch-Collect are directly supplied to the Dispatch task of the second-stage Dispatch-Collect. Specifically, the Apply workload for V_i is encapsulated as a Dispatch task and assigned to (P_i, P_{i+1}) . The input of the Apply computation, i.e., $V_i \boxplus U_{*,i}$, is inherently secret-shared between (P_i, P_{i+1}) due to the Collect of the previous stage of Dispatch-Collect. While the Apply computation aims to perform NN computation on the vertex data, the computation details depend on specific GNN models and the forward or backward layer that is executed. The main output of each Dispatch task is the updated vertex data V'_i secret-shared between (P_i, P_{i+1}) . In the example of Figure 5, there are four Dispatch tasks,

corresponding to the Apply computation of $V_0 \boxplus U_{*,0}$, $V_1 \boxplus U_{*,1}$, $V_2 \boxplus U_{*,2}$, and $V_3 \boxplus U_{*,3}$ respectively.

When a backward GNN layer (suppose it is layer l) is executed, the by-product of each Dispatch task is a gradient D_l^i generated from V_i , also secret-shared between (P_i, P_{i+1}) . The goal of the Collect phase is to aggregate all D_l^i , $\forall i \in [N]$ and produce a global gradient D_l for W_l . Still, since different D_l^i is secret-shared between a different pair of parties (P_i, P_{i+1}) , we have all parties P_i , $i \in [N] \setminus \{0, 1\}$ delegate their shares of D_l^i to P_0 and P_1 . In particular, $\langle D_l^i \rangle_1$ is delegated to P_0 , while $\langle D_l^i \rangle_0$ is delegated to P_1 . (P_0, P_1) then handle the Collect task to aggregate these gradients and apply D_l to optimize W_l . All the data involved in the Collect task is secret-shared between (P_0, P_1) .

Note that, here we select (P_0, P_1) as the delegator parties of the Collect task of this Dispatch-Collect stage. In practice, we can select a different pair of parties as the delegator parties of different Collect tasks to better balance the overhead among all the parties.

5.3 Cascade Iterations by Share Redistribution

The last problem for us is how to cascade iterations, i.e., how to organize the output from the current iteration to serve later iterations. From the above construction, the Dispatch phase of the second-stage Dispatch-Collect secretly shares the updated V'_i between (P_i, P_{i+1}) . However, in the next GAS iteration, V'_i is assigned to V_i and is treated as the input of the Dispatch tasks (in the first-stage Dispatch-Collect) executed by (P_i, P_j) , $j \in [N] \setminus \{i\}$. In other words, V'_i has to be secret-shared between (P_i, P_j) , $j \in [N] \setminus \{i\}$ again. Thus, we have (P_i, P_{i+1}) randomize their shares of V'_i and have P_{i+1} send its share $\langle V'_i \rangle_1 + R_j$ to P_j , $j \in [N] \setminus \{i, i + 1\}$, while having P_i keep $\langle V'_i \rangle_0 - R_j$. We call this process *share redistribution*.

In a backward layer, the optimized W'_l is secret-shared between (P_0, P_1) after the Collect phase of the second-stage Dispatch-Collect. To utilize W'_l in a later forward layer's Dispatch tasks (in the second-stage Dispatch-Collect), we have to perform share redistribution again to make W'_l secret-shared between all pairs of (P_i, P_{i+1}) , $i \in [N]$. Specifically, we have (P_0, P_1) randomize their shares of W'_l and have P_1 send its share $\langle W'_l \rangle_0 + R_i$ to P_i , $i \in [N] \setminus \{0, 1\}$, while having P_0 send its share $\langle W'_l \rangle_1 - R_{i-1}$ to P_i , $i \in [N] \setminus \{0, 1\}$.

6 INSTANTIATE GCN UNDER COGNN

In this section, we provide the details of how we instantiate CoGNN to support the training and inference of Graph Convolutional Network (GCN) models. Specifically, we elaborate on the design of each Dispatch task and Collect task, and how we feed the outcomes of previous tasks to later tasks. The specification of each Dispatch task and Collect task is shown in Protocol 1. The thorough protocol description, i.e., CoGNN-GCN, is provided in Protocol 2.

Notations. We introduce some key notations in protocol description. We use $V.id$ to represent the vertex identifier list of a vertex list, and use $E.src/E.dst$ to represent the source/destination vertex identifier list of an edge list. In addition, we use $V.deg$ to denote the degree list of a vertex list. Accordingly, we use $E.srcDeg/E.dstDeg$ to denote the list of degrees corresponding to the source/destination vertices of an edge list. $X^{-\frac{1}{2}}$ means taking the reciprocal of each element in the list X , and then taking the square root of it.

1	Dispatch-1 ($i, j, \langle V \rangle, \langle E \rangle, V^{\text{dst}}.\text{id} \rightarrow \langle U^{\text{dst}} \rangle$ # for Scatter
2	$\langle V^{\text{src}} \rangle \leftarrow \text{OEP}(\langle V \rangle, V.\text{id}, E.\text{src})$
3	$\langle U \rangle \leftarrow E.\text{srcDeg}^{-\frac{1}{2}} \odot \langle V^{\text{src}} \rangle \odot E.\text{dstDeg}^{-\frac{1}{2}}$
4	$\langle U' \rangle \leftarrow \text{OGA}(\langle U \rangle, E.\text{dst}, +)$
5	$\langle U^{\text{dst}} \rangle \leftarrow \text{OEP}(\langle U' \rangle, E.\text{dst}, V^{\text{dst}}.\text{id})$
6	return $\langle U^{\text{dst}} \rangle$
7	Collect-1 ($i, j, \langle V \rangle, \{\langle U_k^{\text{dst}} \rangle\} \rightarrow \langle V \boxplus U_*^{\text{dst}} \rangle$ # for Gather
8	$\langle V' \rangle \leftarrow \langle V \rangle \odot V.\text{deg}^{-1}$
9	$\langle V \boxplus U_*^{\text{dst}} \rangle \leftarrow \langle V' \rangle + \sum_{k=0}^{N-1} \langle U_k^{\text{dst}} \rangle$
10	return $\langle V \boxplus U_*^{\text{dst}} \rangle$
11	Dispatch-2 ($i, j, l, \text{isForward}, \langle V \rangle, \langle W_l \rangle, \dots, \langle Z \rangle, \langle AH \rangle$ → $\langle \langle V' \rangle, \dots, \langle Z \rangle, \langle \hat{Y} \rangle, \langle D_l \rangle \rangle$ # for Apply
12	if isForward:
13	$\langle Z \rangle \leftarrow \langle V \rangle \cdot \langle W_l \rangle$
14	if $l \neq K-1$: $\langle V' \rangle \leftarrow \text{ReLU}(\langle Z \rangle)$
15	else: $\langle \hat{Y} \rangle \leftarrow \text{Softmax}(\langle Z \rangle)$; $\langle V' \rangle \leftarrow \langle \hat{Y} \rangle - V.y$
16	else:
17	if $l = K-1$: $\langle M \rangle \leftarrow \langle V \rangle$
18	else: $\langle M \rangle \leftarrow \text{ReLU}'(\langle Z \rangle) \odot \langle V \rangle$
19	$\langle D_l \rangle \leftarrow \langle AH \rangle \cdot \langle M \rangle$
20	if $l \neq 0$: $\langle V' \rangle \leftarrow \langle M \rangle \cdot \langle W_l \rangle$
21	return $\langle V' \rangle, \dots, \langle Z \rangle, \langle \hat{Y} \rangle, \langle D_l \rangle$
22	Collect-2 ($i, j, \langle W_l \rangle, \{\langle D_l^k \rangle\}, \{ V_k \} \rightarrow \langle W_l' \rangle$ # for Gradient
23	$\langle D_l \rangle \leftarrow \frac{\sum_{k=0}^{N-1} (V_k \cdot \langle D_l^k \rangle)}{\sum_{k=0}^{N-1} V_k }$ # Aggregation
24	$\langle W_l' \rangle \leftarrow \text{Opt}(\langle W_l \rangle, \langle D_l \rangle)$ # Application
25	return $\langle W_l' \rangle$

Protocol 1: Dispatch tasks and Collect tasks in the first-stage and the second-stage Dispatch-Collect.

6.1 Graph Data Secret Share

Each P_i breaks its V_i into 2-out-of-2 additive secret shares, keeps one share as secret, and distributes the other share to all the other parties. At the same time, E_i and $E_{i,j}, \forall j \in [N] \setminus \{i\}$ are also broken into secret shares. However, unlike V_i , E_i is only shared with its neighboring (next) party P_{i+1} . $E_{i,j}$ is only shared with party P_j .

Initially, V_i stores $H_0^i = X_i$, i.e., the features of vertices. For simplicity, the input preparation process is omitted in Protocol 2.

6.2 The First-Stage Dispatch-Collect

6.2.1 Dispatch Task. A Dispatch task in the first-stage Dispatch-Collect (denoted as Dispatch-1) is executed by two parties (P_i, P_j) , $i, j \in [N]$, $i \neq j$ and performs Scatter for GCN. The inputs of Dispatch-1 include a vertex list $\langle V \rangle$ and an edge list $\langle E \rangle$. The output is an update list $\langle U^{\text{dst}} \rangle$. All these data is secret-shared between (P_i, P_j) .

Based on our message passing mechanism introduced in § 5.1, in this Dispatch-1, the Scatter for GCN is executed in four steps.

- (1) Use OEP to construct a source vertex list $\langle V^{\text{src}} \rangle$ corresponding to $E.\text{src}$, as line 2 of Protocol 1. Both $V.\text{id}$ and $E.\text{src}$ are supplied by P_i .
- (2) $\langle V^{\text{src}} \rangle$ is then used to perform element-wise computation with $\langle E \rangle$, to generate an update list of the same length as $\langle E \rangle$, as line 3 of Protocol 1. $E.\text{srcDeg}$ and $E.\text{dstDeg}$ are supplied by

either P_i or P_j , depending on who holds them. $\odot$ represents element-wise multiplication between the two lists.

- (3) Merge the updates in $\langle U \rangle$ that share the same destination vertices. According to CoGNN's requirement of the input edge list order (see § 4), all the updates targeting the same destination vertex are already placed in a contiguous segment. So we invoke OGA for this, as line 4 of Protocol 1. The operation to merge two updates is an addition, i.e., +.
- (4) Use OEP to extract the merged updates and place them at the same positions as their destination vertices, as line 5 of Protocol 1. Note that V^{dst} represents the destination vertex list of E . $V^{\text{dst}}.\text{id}$ is supplied by either P_i or P_j , depending on who owns it. Some identifiers in $V^{\text{dst}}.\text{id}$ might not have an equal index in $E.\text{dst}$, and OEP sets their corresponding elements in $\langle U^{\text{dst}} \rangle$ to 0.

The Dispatch-1 tasks for a forward layer and a backward layer of GCN are the same. The only difference is in the data stored in the vertex list. In a forward layer, the vertex list stores the features or hidden representation of each vertex. In a backward layer, the vertex list stores a back-propagated intermediate gradient value. See Protocol 2 for the detailed inputs fed to each Dispatch-1 task.

6.2.2 Collect Task. In an N -party setting, the Collect-1 tasks are delegated to N pairs of parties, i.e., $(P_0, P_1), (P_1, P_2), \dots, (P_{N-1}, P_0)$. Specifically, the Collect-1 task of (P_i, P_{i+1}) deals with the Gather of V_i . For V_i , suppose that the parties have already delegated their shares of update lists to P_{i+1} . Now (P_i, P_{i+1}) hold N secret-shared update lists, i.e., $\langle U_{0,i}^{\text{dst}} \rangle, \langle U_{1,i}^{\text{dst}} \rangle, \dots, \langle U_{N-1,i}^{\text{dst}} \rangle$. The goal of the Collect-1 task is to merge all these update lists with $\langle V_i \rangle$.

Thus, the inputs of the Collect-1 task are composed of a set of update lists $\{\langle U_k^{\text{dst}} \rangle\}$ and a vertex list $\langle V \rangle$. The output of the vertex list which has merged all the update lists, i.e., $\langle V \boxplus U_*^{\text{dst}} \rangle$.

For GCN, the Collect-1 task is executed in two steps:

- (1) Scale the vertex list according to the degree of each vertex, as line 8 of Protocol 1, where $V.\text{deg}$ is supplied by P_i .
- (2) Add all the update lists to the vertex list, as line 9 of Protocol 1.

The Collect-1 tasks for forward and backward layers are the same. See Protocol 2 for the inputs fed to each Collect-1 task.

6.3 The Second-Stage Dispatch-Collect

6.3.1 Dispatch Task. In the second-stage Dispatch-Collect, each Dispatch-2 task performs the Apply computation for a $V_i, i \in [N]$ and is handled by a pair of parties (P_i, P_{i+1}) . Since the Apply computations for different forward/backward layers differ, the Dispatch tasks are also different. Suppose that the GCN has K layers of NN computations. In total, there are five situations for the Dispatch-2 task, i.e., when the Dispatch-2 task is in a forward layer rather than the last layer (i.e., layer $K-1$), in the last forward layer, in the first backward layer (i.e., layer $K-1$), in the last backward layer (i.e., layer 0) and in the rest of the backward layers.

The main inputs of the Dispatch-2 task include a vertex list merged with updates, denoted as $\langle V \rangle$, and a weight matrix, denoted as $\langle W_l \rangle$. The main output of the Dispatch-2 task is the updated vertex list $\langle V' \rangle$. We discuss the above situations as follows:

Protocol CoGNN-GCN	
<i>Input.</i> $P_i, \forall i \in [N]$ provides $E_i, E_{i,j}$ and $V_i = H_0^i = X_i$. Negotiated $\text{alg} = (K, \text{maxEp}), \{W_l \mid l \in [K]\}$. <i>Output.</i> $P_i, \forall i \in [N]$ gets $V'_i = P_i$ and $\{W'_l \mid l \in [K]\}$.	
1	for $ep \in [\text{maxEp}]$:
2	for $l \in [K]$: # Forward Pass , iterate from layer 0 to $K - 1$
3	for $i \in [N]$: $\langle V_i \rangle \leftarrow \langle H_l^i \rangle$ # Set the layer input (vertex data)
4	for $i, j \in [N], i \neq j$: # Scatter vertex embedding using Dispatch-1
5	$\langle U_{i,j}^{\text{dst}} \rangle \leftarrow \text{Dispatch-1}(i, j, \langle V_i \rangle, \langle E_{i,j} \rangle, V_j.\text{id})$; if $j = i + 1$: $\langle U_{i,i}^{\text{dst}} \rangle \leftarrow \text{Dispatch-1}(i, i + 1, \langle V_i \rangle, \langle E_i \rangle, V_i.\text{id})$
6	for $i \in [N]$:
7	for $j \in [N] \setminus \{i, i + 1\}$: P_j sends $\langle U_{j,i}^{\text{dst}} \rangle_1$ to P_{i+1} # Delegate secret shares
8	$\langle V_i \boxplus U_{*,i}^{\text{dst}} \rangle \leftarrow \text{Collect-1}(i, i + 1, \langle V_i \rangle, \{\langle U_{k,i}^{\text{dst}} \rangle\})$ # Gather using Collect-1
9	if $i \neq K - 1$: $\langle V'_i \rangle, \langle Z_l^i \rangle \leftarrow \text{Dispatch-2}(i, i + 1, l, \text{true}, \langle V_i \boxplus U_{*,i}^{\text{dst}} \rangle, \langle W_l \rangle)$ # Forward NN computation using Dispatch-2
10	else : $\langle V'_i \rangle, \langle Z_l^i \rangle, \langle P_i \rangle \leftarrow \text{Dispatch-2}(i, i + 1, l, \text{true}, \langle V_i \boxplus U_{*,i}^{\text{dst}} \rangle, \langle W_l \rangle)$ # Forward prediction using Dispatch-2
11	for $j \in [N] \setminus \{i, i + 1\}$: P_{i+1} sends $\langle V'_i \rangle_1 + R_j$ to P_j # Redistribute secret shares
12	$\langle H_{l+1}^i \rangle \leftarrow \langle V'_i \rangle$; $\langle \hat{A}H_l^i \rangle \leftarrow \langle V_i \boxplus U_{*,i}^{\text{dst}} \rangle$ # Set the layer output
13	for $l \in [K]$ (reverse): # Backward Pass , iterate from layer $K - 1$ to 0
14	for $i \in [N]$: # Set the layer input (vertex data)
15	if $l = K - 1$: $\langle V_i \rangle \leftarrow \langle H_K^i \rangle$; else : $\langle V_i \rangle \leftarrow \langle G_l^i \rangle$
16	for $i, j \in [N], i \neq j$: # Scatter intermediate gradients using Dispatch-1
17	$\langle U_{i,j}^{\text{dst}} \rangle \leftarrow \text{Dispatch-1}(i, j, \langle V_{i,j} \rangle, \langle E_{i,j} \rangle, V_j.\text{id})$; if $j = i + 1$: $\langle U_{i,i}^{\text{dst}} \rangle \leftarrow \text{Dispatch-1}(i, i + 1, \langle V_i \rangle, \langle E_i \rangle, V_i.\text{id})$
18	for $i \in [N]$:
19	for $j \in [N] \setminus \{i, i + 1\}$: P_j sends $\langle U_{j,i}^{\text{dst}} \rangle_1$ to P_{i+1} # Delegate secret shares
20	$\langle V_i \boxplus U_{*,i}^{\text{dst}} \rangle \leftarrow \text{Collect-1}(i, i + 1, \langle V_i \rangle, \{\langle U_{k,i}^{\text{dst}} \rangle\})$ # Gather using Collect-1
21	if $l = K - 1$: $\langle V_i \boxplus U_{*,i}^{\text{dst}} \rangle \leftarrow \langle V_i \rangle$ # For layer $K - 1$, $\langle V_i \boxplus U_{*,i}^{\text{dst}} \rangle$ is directly set to $\langle V_i \rangle$
22	if $i \neq 0$: $\langle V'_i \rangle, \langle D_l^i \rangle \leftarrow \text{Dispatch-2}(i, i + 1, l, \text{false}, \langle V_i \boxplus U_{*,i}^{\text{dst}} \rangle, \langle W_l \rangle, \langle Z_l^i \rangle, \langle (\hat{A}H_l^i)^T \rangle)$
23	else : $\langle D_l^i \rangle \leftarrow \text{Dispatch-2}(i, i + 1, l, \text{false}, \langle V_i \boxplus U_{*,i}^{\text{dst}} \rangle, \langle W_l \rangle, \langle Z_l^i \rangle, \langle (\hat{A}H_l^i)^T \rangle)$ # Gradient computation using Dispatch-2
24	for $i \in [N] \setminus \{0, 1\}$: P_i sends $\langle D_l^i \rangle_0$ to P_1 and sends $\langle D_l^{j-1} \rangle_1$ to P_0 # Delegate secret shares of gradients
25	$\langle W'_l \rangle \leftarrow \text{Collect-2}(0, 1, \langle W_l \rangle, \{\langle D_l^k \rangle\}, \{V_k\})$, $k \in [N]$ # Gradient aggregation and application using Collect-2
26	for $i \in [N] \setminus \{0, 1\}$: P_1 sends $\langle W'_l \rangle_0 + R_i$ to P_i ; P_0 sends $\langle W'_l \rangle_1 - R_{i-1}$ to P_i # Redistribute secret shares of the new weight
27	for $i \in [N] \setminus \{0, 1\}$: $\langle G_{l-1}^i \rangle \leftarrow \langle V'_i \rangle$; $\langle W_l \rangle \leftarrow \langle W'_l \rangle$ # Set the layer output

Protocol 2: The construction of CoGNN-GCN.

- When the Dispatch-2 task is in a forward layer rather than the last layer, the Dispatch task executes weight multiplication and *ReLU* activation, as lines 13 and 14 of Protocol 1.
- When the Dispatch-2 task is in the last forward layer, it executes weight multiplication and *Softmax* normalization, as lines 13, 15 of Protocol 1. Here the additional output produced by the Dispatch task is the prediction list $\langle \hat{Y} \rangle$ for V . $V.y$ denotes the label list for V . $\langle \hat{Y} \rangle - V.y$ is the gradient of the cross-entropy loss w.r.t. $\langle Z \rangle$.
- When the Dispatch-2 task is in a backward layer rather than the first and the last backward layer, it executes lines 18, 19 and 20 of Protocol 1. Note that both $\langle Z \rangle$ and $\langle \hat{A}H \rangle$ are intermediate computation results from forward layers and they serve as additional inputs to the Dispatch-2 task here.
- When the Dispatch-2 task is in the first backward layer, it executes lines 17, 19 and 20 of Protocol 1.
- When the Dispatch-2 task is in the last backward layer, it executes lines 18, 19 of Protocol 1.

In Protocol 1, we use ... to separate the required inputs/outputs and the optional inputs/outputs of Dispatch-2. The data stored in $\langle V \rangle$ and $\langle V' \rangle$ has different meanings in different GCN forward/backward

layers. See Protocol 2 for the details of how CoGNN prepares inputs for Dispatch-2 tasks of different layers.

6.3.2 Collect Task. In a backward layer, the second-stage Dispatch-Collect has a Collect phase (corresponding to the Collect-2 task). The goal of the Collect-2 task is to aggregate the gradient $\langle D_l^i \rangle$ produced on different $V_i, i \in [N]$, and to optimize the weight matrix $\langle W_l \rangle$ using the aggregated gradient. Suppose that all the secret shares of different $\langle D_l^i \rangle$ have been delegated to (P_0, P_1) according to the specification in § 5.2.2. The inputs of the Collect-2 task include a set of secret-shared gradients $\{\langle D_l^k \rangle\}, k \in [N]$ and the weight matrix $\langle W_l \rangle$. The output is the optimized weight matrix $\langle W'_l \rangle$.

The Collect-2 task consists of two steps:

- (1) Obtain the global gradient by performing a weighted sum for all $\{\langle D_l^k \rangle\}$, as line 23 of Protocol 1.
- (2) Optimize $\langle W_l \rangle$ using $\langle D_l \rangle$ as line 24 of Protocol 1. Here *Opt* abbreviates the weight optimizing process.

6.4 End-to-End Training and Inference

Training. As shown in line 3 Protocol 2, at the beginning of each forward layer l , the vertex data $\langle V_i \rangle$ of P_i is set to the hidden representation $\langle H_l^i \rangle$. For layer 0, $H_0^i = X_i$. $\langle V_i \rangle$ are fed to the Dispatch-1 tasks to compute update vectors $\langle U_{j,i}^{\text{dst}} \rangle, j \in [N]$. After that, the Collect-1 task aggregates all $\langle U_{j,i}^{\text{dst}} \rangle, j \in [N]$ to $\langle V_i \rangle$, and obtain $\langle V_i \boxplus U_{*,i}^{\text{dst}} \rangle$. Then, the Dispatch-2 task performs NN-related computations on $\langle V_i \boxplus U_{*,i}^{\text{dst}} \rangle$ to obtain $\langle V_i' \rangle$. If it is the last layer $K-1$, $\langle H_K^i \rangle = \langle V_i' \rangle$ stores $\langle \hat{Y}_i \rangle - V_i.y$. Note that $\langle V_i \boxplus U_{*,i}^{\text{dst}} \rangle$ is recorded as $\langle \hat{A}H_l^i \rangle$. Both $\langle \hat{A}H_l^i \rangle$ and $\langle Z_l^i \rangle$ are stored as forward layer l 's results and used in the corresponding backward layer's computation.

The backward computation iterates from layer $K-1$ to layer 0. At the beginning of computing each backward layer, the data assigned to $\langle V_i \rangle$ depends on the layer index l . If it is layer $K-1$, i.e., the first backward layer, $\langle V_i \rangle$ gets $\langle H_K^i \rangle$, i.e., $\langle \hat{Y}_i \rangle - V_i.y$. Otherwise, $\langle V_i \rangle$ gets $\langle G_l^i \rangle$. Note that, when $l = K-1$, there is no need to execute Dispatch-1 tasks and the Collect-1 task on $\langle V_i \rangle$. We directly assign $\langle V_i \rangle$ to $\langle V_i \boxplus U_{*,i}^{\text{dst}} \rangle$, as line 21 of Protocol 2. For Dispatch-2 tasks in backward layers, if $l \neq 0$, each Dispatch-2 task produces $\langle V_i' \rangle$ and a gradient $\langle D_l^i \rangle$. If $l = 0$, each Dispatch-2 task only produces a gradient $\langle D_l^i \rangle$. Lines 24 to 26 in Protocol 2 show how the shares of all $\langle D_l^i \rangle$ are delegated to (P_0, P_1) for gradient aggregation and weight optimization in the Collect-2 task, and how the aggregated result is redistributed back to all $P_i, i \in [N] \setminus \{0, 1\}$. Line 27 of Protocol 2 assigns $\langle V_i' \rangle$ to $\langle G_{l-1}^i \rangle$ and assigns $\langle W_l' \rangle$ to $\langle W_l \rangle$. $\langle G_{l-1}^i \rangle$ is used for computing the next backward layer, while $\langle W_l \rangle$ is used in the next-epoch computation.

Inference. In CoGNN, performing a GCN inference process requires only a single run of the **Forward** part in Protocol 2. The weight matrices $\{W_l\}, l \in [K]$ can be either held by all parties in plaintext, or secretly shared as generated by the CoGNN training process, i.e., each pair of parties (P_i, P_{i+1}) hold a different two-party secret share of $\{W_l\}$.

Analysis. In total, running each forward layer requires N^2 Dispatch-1 tasks, N Collect-1 tasks and N Dispatch-2 tasks. Running the first backward layer ($l = K-1$) requires N Dispatch-2 tasks and one Collect-2 task, while running each of the rest of the backward layers needs N^2 Dispatch-1 tasks, N Collect-1 tasks, N Dispatch-2 tasks and one Collect-2 task. In computing one forward or backward layer, tasks of the same type are executed distributedly (across all parties) and concurrently in CoGNN, making it scalable as more parties are involved. In § 8.2.2, we show that CoGNN is highly scalable: when the number of graph owners increases, the per-party running duration and communication grows modestly.

6.5 Result Reconstruction

Once the GCN training or inference process finishes, the parties jointly reconstruct the secret-shared computation outputs. For all forward layers, the final output is the updated vertex data $\langle V_i' \rangle$ storing the prediction values $\langle \hat{Y}_i \rangle$. For each backward layer l , the output is the updated model weight $\langle W_l' \rangle$. Both $\langle V_i' \rangle$ and $\langle W_l' \rangle$ are secret-shared between $(P_i, P_{i+1}), i \in [N]$. As a result, P_{i+1} sends $\langle V_i' \rangle_1$ and $\{\langle W_l' \rangle_1\}, l \in [K]$ to P_i for reconstructing V_i' and $\{W_l'\}, l \in [K]$ respectively. This part is omitted in Protocol 2.

6.6 Further Optimizations

Optimization w.r.t. the Input Dimension. The first optimization factor we consider here is the complexity of CoGNN w.r.t. the input dimension (or element dimension) of each layer. Our consideration is out of the fact that the input and output dimensions of different GCN layers are quite uneven. For example, for a typical two-layer GCN, the feature dimension could be as high as hundreds or even thousands, while the hidden representation dimensions are typically less than one hundred, and the number of classes is typically less than ten. Since the overall computation/communication overhead of a GCN layer is linear to its input dimension, the overhead of GCN layers with a high input dimension is also high.

However, we find that much overhead caused by the high input dimensions can be well circumvented by slightly changing Protocol 2. Concretely, in our current specification of CoGNN-GCN, the execution of each forward layer (as specified by Equation (1)) begins with performing message passing (i.e., $\hat{A}H_l$) using Scatter-Gather, and then handles NN-related computation, i.e., applying W_l and σ . The problem is that the dimensions of the elements participating in messaging passing are extremely high. For example, at layer 0, we perform message passing for each sample's high-dimension features. This problem can be alleviated by the following transformation to Equation (1):

$$H_{l+1} = \sigma(\hat{A}(H_l W_l)). \quad (6)$$

In other words, we can advance part of the computation conducted during the Apply phase, i.e., weight matrix multiplication, to the front of the Scatter phase (denoted as **PreScatter**), when the output dimension is lower than the input dimension. Such transformation largely slashes the overhead of Scatter-Gather for forward layers. We do not apply the transformation to backward layers since the output dimensions are typically higher than that of the input.

The problem here is that in backward layers, to derive the gradient w.r.t. the weight matrix (see Equation (2)), we need $\hat{A}H_{l-1}$, which is not computed due to our aforementioned transformation. Our workaround is the following transformation to Equation (2):

$$D_{l-1} = H_{l-1}^T (\hat{A}^T (\sigma'(Z_{l-1}) \odot \hat{A}^T G_l)). \quad (7)$$

In CoGNN, this transformation corresponds to two executions of Scatter-Gather (i.e., Dispatch-1 and Collect-1) for each backward layer. As long as the corresponding forward layer's input dimension is higher than two times its output dimension, the transformation still brings optimization.

Optimization w.r.t. $|E|$. In CoGNN, the overhead of the Scatter phase is linear to $|E|$. Though this complexity is inherent to GNN computation (since we have to do message passing on each edge), we can advance some costly computations to the front of Scatter or delay them to the back of Gather, in order to make their overhead linear to $|V|$. The exact benefit is that the concrete overhead of the overall message passing process is largely reduced. Specifically, in GCN, we make the following further transformation to Equation (6).

$$H_{l+1} = \sigma(\tilde{D}^{-\frac{1}{2}} (\tilde{A} (\tilde{D}^{-\frac{1}{2}} (H_l W_l)))). \quad (8)$$

Previously, in Equation (6), $\hat{A}$ is a weighted matrix. These weights are represented as edge weights under the CoGNN computation model. As a result, the vectorized Scatter computation includes the

multiplication with the edge weights at each vector slot, which is costly due to linear complexity w.r.t. $|E|$. After the above transformation, since $\tilde{D}^{-\frac{1}{2}}$ is a diagonal matrix, multiplying $\tilde{D}^{-\frac{1}{2}}$ to $H_l W_l$ means multiplying each row of $H_l W_l$ with a scalar, which only has a linear cost w.r.t. the length H_l , i.e., $|V|$. On the other hand, since $\tilde{A}$ is an unweighted adjacency matrix containing elements valued at $\{0, 1\}$, the whole Scatter-Gather phase now contains no multiplications. To achieve this transformation in CoGNN, we advance the inner multiplication with $\tilde{D}^{-\frac{1}{2}}$ to the front of the Scatter phase (i.e., right after executing $H_l W_l$ in **PreScatter**), and delay the outer multiplication with $\tilde{D}^{-\frac{1}{2}}$ to the back of the Gather phase (denoted as **PostGather**).

Experimental results in § 8.2.2 show that these two optimizations bring 13 ~ 21x improvement to the per-epoch duration of CoGNN, for various numbers of graph owners.

7 SECURITY THEOREM

THEOREM 1. *Our CoGNN-GCN protocol (as specified in Protocol 2) securely realizes the functionality $\mathcal{F}_{\text{CoGNN}}$ in the $(\mathcal{F}_{\text{Dsp-1}}, \mathcal{F}_{\text{Dsp-2}}, \mathcal{F}_{\text{Clic-1}}, \mathcal{F}_{\text{Clic-2}})$ -hybrid model against a semi-honest, non-uniform adversary $\mathcal{A}$ corrupting one party at a time. Formally, for every PPT, semi-honest and non-uniform adversary $\mathcal{A}$ that corrupts one party P_i ($i \in [N]$), there exists a PPT, non-uniform simulator $\mathcal{S}$ corrupting the same party in the ideal world of $\mathcal{F}_{\text{CoGNN}}$, which satisfies:*

$$\begin{aligned} \text{REAL}_{\text{CoGNN-GCN}, \mathcal{A}}^{\mathcal{F}_{\text{Dsp-1}}, \mathcal{F}_{\text{Dsp-2}}, \mathcal{F}_{\text{Clic-1}}, \mathcal{F}_{\text{Clic-2}}}(\kappa, \{E_i\}, \{E_{i,j}\}, \{V_i\}, \{W_l\}, \text{alg}) \\ \stackrel{c}{\equiv} \text{IDEAL}_{\mathcal{F}_{\text{CoGNN}}, \mathcal{S}}(\kappa, \{E_i\}, \{E_{i,j}\}, \{V_i\}, \{W_l\}, \text{alg}), \end{aligned}$$

where $\text{REAL}_{\text{CoGNN-GCN}, \mathcal{A}}^{\mathcal{F}_{\text{Dsp-1}}, \mathcal{F}_{\text{Dsp-2}}, \mathcal{F}_{\text{Clic-1}}, \mathcal{F}_{\text{Clic-2}}}$ represents a joint distribution over the view of the adversary (the corrupted party's input, randomness, protocol transcript) and the protocol output, when P_i and $P_j, \forall j \in [N]$ interact in the $(\mathcal{F}_{\text{Dsp-1}}, \mathcal{F}_{\text{Dsp-2}}, \mathcal{F}_{\text{Clic-1}}, \mathcal{F}_{\text{Clic-2}})$ -hybrid CoGNN on inputs $(\{E_i\}, \{E_{i,j}\}, \{V_i\}, \{W_l\}, \text{alg})$ and computational security parameter κ . $(\mathcal{F}_{\text{Dsp-1}}, \mathcal{F}_{\text{Dsp-2}}, \mathcal{F}_{\text{Clic-1}}, \mathcal{F}_{\text{Clic-2}})$ are the functionalities for (Dispatch-1, Dispatch-2, Collect-1, Collect-2) respectively. $\text{IDEAL}_{\mathcal{F}_{\text{CoGNN}}, \mathcal{S}}(\kappa, \{E_i\}, \{E_{i,j}\}, \{V_i\}, \{W_l\}, \text{alg})$ represents a joint distribution over the simulated view of the corrupted party and the functionality output, when P_i and $P_j, \forall j \in [N]$ interact with $\mathcal{F}_{\text{CoGNN}}$ on inputs $(\{E_i\}, \{E_{i,j}\}, \{V_i\}, \{W_l\}, \text{alg})$ and computational security parameter κ ; and $\stackrel{c}{\equiv}$ means the two distributions are computationally indistinguishable (in κ).

Theorem 1 can be understood through two intuitions: (i) the pairwise computations are all constructed using provably secure gadgets, including generic 2PC, OEP and OGA; (ii) the multi-party coordination is based on share delegation/redistribution, where all the messages received by each party are independent and uniformly random. More specifically, the security of CoGNN-GCN stems from the security of $(\mathcal{F}_{\text{Dsp-1}}, \mathcal{F}_{\text{Dsp-2}}, \mathcal{F}_{\text{Clic-1}}, \mathcal{F}_{\text{Clic-2}})$. Among them, the security of $(\mathcal{F}_{\text{Dsp-2}}, \mathcal{F}_{\text{Clic-1}}, \mathcal{F}_{\text{Clic-2}})$ is straightforward since their realizations include only simple 2PC operations, e.g., addition, multiplication, ReLU and Softmax, as shown in Protocol 1. On the other hand, the security of $\mathcal{F}_{\text{Dsp-1}}$ relies on the secure realizations of $\mathcal{F}_{\text{OEP}}$ and $\mathcal{F}_{\text{OGA}}$. We provide the detailed construction of these two protocols, their security lemmas, and the detailed proof of Theorem 1 in our technical report [58].

8 IMPLEMENTATION & EVALUATION

8.1 Implementation

The core component of our CoGNN prototype¹ is a GNN execution engine (4044 LoC C++) fulfilling the CoGNN framework. We further realize the OEP and OGA protocols (843 LoC C++) and integrate them into the engine for efficient Scatter-Gather computation. Based on that, we implement and optimize GCN (1922 LoC C++) to evaluate the concrete efficiency of CoGNN. Specifically, we use the RLWE-based HE and polynomial vector encoding provided by troy [4, 30] (A GPU-accelerated HE library) to support the batch HE operation of OEP² and the matrix multiplication between secret-shared matrices. The OT-related operations required by OEP and OGA are supported by the libOTe [2] library's implementation of Silent OT [10]. The rest of the computations on secret-shared data, e.g., vector-scalar multiplication, *Softmax* and *ReLU*, are implemented based on the SCI library [3]. We replace SCI's OT implementation with libOTe's for better efficiency.

8.2 Evaluation

Our evaluations center around the following:

- **Efficiency and Scalability.** In § 8.2.2, we conduct an extensive comparison between CoGNN and the SOTA SML-based approach [26] to demonstrate CoGNN's efficiency improvement, as well as its scalability for an increasing number of graph owners. The comparison includes end-to-end duration/communication measurements on various datasets;
- **Accuracy.** In § 8.2.3, we show that CoGNN yields model accuracy comparable to plaintext GNN, while significantly surpassing prior FL-based GNN. This part of the experimental results reveals the importance of considering inter-edges during GNN collaboration.

Additionally, in § 8.2.4, we present the breakdowns of the optimized CoGNN's running durations to quantify the effectiveness of our message passing mechanism. Due to the page limit, we defer several supporting evaluation results to our technical report [58], including the comparison between our message passing approach and prior art, and CoGNN's duration breakdowns before optimizations and communication statistics for training/inference.

8.2.1 Setup. Testbed. Our testbed is a Linux server with two Intel (R) Xeon (R) Gold 6348 CPUs at 2.60GHz. The network environment is simulated using Linux network namespace and the tc command. We place different parties in different network namespaces connected via a virtual bridge, and use the tc command to set up specific bandwidth and latency. In particular, all our efficiency tests run in a simulated LAN environment with 4Gbps bandwidth and 1ms latency. A computation task assigned to each pair of parties (i.e., P_i and $P_j, i \neq j$) runs in a single thread.

Dataset. We evaluate CoGNN on three graph datasets commonly used for vertex-level prediction tasks, i.e., Cora [34], CiteSeer [16], PubMed [45]. During GNN training, we consider the semi-supervised setting, where only a relatively small proportion of vertices have labels but all vertices have features and participate in the training process. We perform batch gradient descent using the full graph

¹<https://github.com/InspiringGroup-Lab/CoGNN>

²We use HE for the offline phase of OEP. See our technical report for the details.

dataset for every training iteration (epoch). The vertices with labels are treated as the training set, while the rest of the vertices are split into validation and test sets. Considering the number of vertices varies a lot for the three datasets, we use different train/valid/test splits for them. To distribute the whole graph dataset evenly to all the graph owners, we use random graph partition, i.e., uniformly randomly assigning each vertex to one of all the graph owners.

To evaluate the *efficiency and scalability* of CoGNN, we split each dataset into 5 partitions, each partition corresponding to a graph owner. As the number of graph owners increases, more partitions are involved in collaborative training, and the scale of the global graph also increases. To evaluate the *accuracy* of CoGNN, we split all three datasets into different numbers of partitions, and involve all partitions in training. In other words, the scale of the global graph is fixed, while the number of graph owners changes. In this setting, as the number of graph owners increases, the number of inter-edges also increases.

In our technical report, we provide additional information regarding the three datasets, the train/valid/test splits we used, and the numbers of inter-edges for different numbers of graph owners.

Parameter. For the cryptographic part, we set the ring length L of fixed-point encoding to 2^{64} and the scaler for encoding a floating point number to 2^{13} . The parameters set for the RLWE-based HE scheme are $\{N = 8192, p = 2^{44}, q \approx 2^{180}\}$. It means each encrypted number is encoded over the ring of $\mathbb{Z}_{2^{44}}$ (i.e., the plaintext space is $\mathbb{Z}_{2^{44}}$). We use the extend and truncate operations provided by the SCI library to switch between the rings of $\mathbb{Z}_{2^{64}}$ and $\mathbb{Z}_{2^{44}}$. For the model structure, we use a 2-layer GCN and set the hidden layer dimension to 16. The first layer is activated by *ReLU*, while the second layer is normalized by *Softmax*. We use cross-entropy loss and full-batch gradient descent. For the model training part, we set the number of epochs to 90. The learning rates for Cora/CiteSeer/PubMed are set to 0.5/0.4/8.0 to guarantee that CoGNN and all the baseline methods have converged (but not overfitted) after the 90 epochs. The model weights are randomly initialized using the Glorot's method [17]. For FedGNN and CoGNN, we take the average of their accuracy over the test set of each party as their final test accuracy.

8.2.2 Scalability & Efficiency. Since the source code and evaluation details of the SOTA SML-based approach [26] (denoted as GraphSC here since it is based on the GraphSC paradigm) are unavailable, for fair comparison, we re-produce it using the same MPC, OT and HE libraries as used in our CoGNN prototype. We use our OEP protocol for the shuffle operations in GraphSC, and use prefix network construction to realize the *propagate* operation for Scatter and *aggregate* operation for Gather. All these realization details try to achieve the best efficiency for GraphSC.

End-to-end Efficiency and Scalability. We compare CoGNN and GraphSC under different numbers of graph owners. According to our evaluation setup, as the number of graph owners increases, the scale (i.e., numbers of edges and vertices) of the global graph increases linearly. The number of computing parties in CoGNN is the same as the number of graph owners, while the number of computing parties in GraphSC remains unchanged and is always 2. We set the same number of threads between each pair of parties for CoGNN

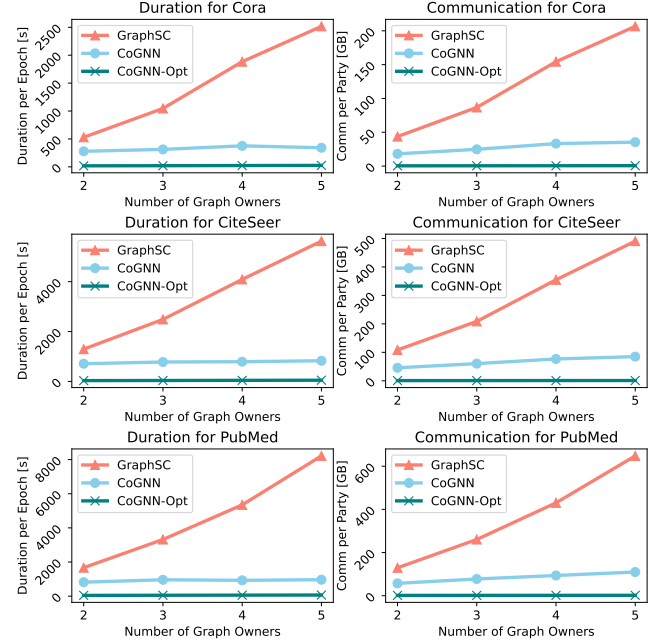


Figure 6: Duration and communication per epoch of each computing party for different numbers of graph owners.

and GraphSC. Figure 6 shows the running duration and communication (send + receive) of each computing party in one epoch of full-batch training. The three lines in each subfigure represent GraphSC, CoGNN, and CoGNN with optimizations (i.e., CoGNN-Opt) respectively. CoGNN shows significant elevation over GraphSC in both efficiency and scalability. Specifically, compared to GraphSC when there are 2 graph owners, CoGNN (CoGNN-Opt) reduces the running duration by 1.9 ~ 2.0x (29 ~ 39x), and reduces the per-party communication of by 2.2 ~ 2.4x (88 ~ 164x). The duration/communication improvements are enlarged as the number of graph owners increases. In particular, when the number of graph owners increases from 2 to 5, the per-party duration (communication) of GraphSC is 4.3 ~ 4.9 (4.5 ~ 5.0) times higher. In contrast, the per-party duration (communication) grows to 1.1 ~ 1.2x (1.8 ~ 1.9x) in CoGNN, and grows to 1.4 ~ 1.5x (1.2 ~ 1.5x) in CoGNN-Opt. The per-party duration/communication of CoGNN grows much slower than GraphSC, indicating CoGNN's better scalability as more graph owners are involved. Compared to GraphSC when there are 5 graph owners, CoGNN (CoGNN-Opt) reduces the running duration by 6.7 ~ 8.5x (97 ~ 123x), and reduces the per-party communication by 5.8 ~ 5.9x (323 ~ 522x).

8.2.3 Model Accuracy. We compare the accuracy of CoGNN and the FL-based approach (abbreviated as FedGNN) to concretely show the benefits of global GCN training *with* inter-edge message passing. The accuracy baseline for us is plaintext GCN training on the global graph, denoted as PlainGNN. For FedGNN, we use the common scheme named *FedAvg* [51], which is equivalent to the gradient aggregation in CoGNN. Both the PlainGNN and FedGNN schemes we compare use the same training hyperparameters as CoGNN. FedGNN uses the same graph partitions as CoGNN.

We compare the accuracy of PlainGNN, CoGNN and FedGNN for different numbers of parties. According to our dataset setup,

Dataset	OEP Pre	PreScatter	Scatter OEP (1)	Scatter Comp	Scatter OGA	Scatter OEP (2)	Gather	PostGather	Apply	Total
Cora	0.33	9.82	0.13	-	1.10	0.01	0.01	4.20	17.29	32.88
CiteSeer	0.35	25.64	0.13	-	1.06	0.01	0.01	4.52	34.18	65.89
PubMed	2.69	29.11	0.26	-	2.61	0.05	0.01	14.51	44.84	94.08

Table 1: Execution duration [s] breakdown for each full-batch training epoch of CoGNN with optimization.

Dataset	OEP Pre	PreScatter	Scatter OEP (1)	Scatter Comp	Scatter OGA	Scatter OEP (2)	Gather	PostGather	Apply	Total
Cora	0.09	6.97	0.08	-	0.61	0.01	0.01	2.80	7.12	17.68
CiteSeer	0.08	22.62	0.08	-	0.61	0.01	0.01	3.37	7.50	34.27
PubMed	0.55	17.04	0.17	-	1.37	0.03	0.01	12.71	20.93	52.79

Table 2: Execution duration [s] breakdown for each full-batch inference of CoGNN with optimization.

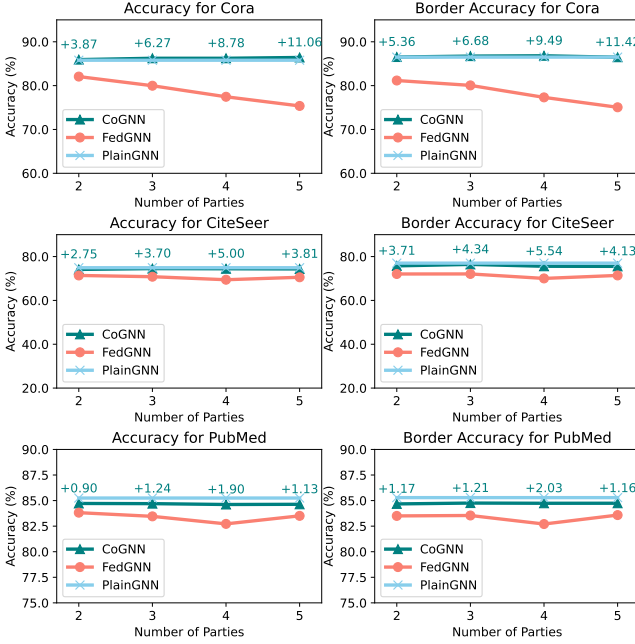


Figure 7: Accuracy for different numbers of parties. (+) represents the accuracy difference between CoGNN and FedGNN.

the number of inter-edges increases as the number of parties increases, meaning that more edges are ignored by FedGNN. We are also interested in vertices connected by inter-edges. We call them *border vertices* and call their accuracy as *border accuracy*. Figure 7 shows that, (i) the accuracy and border accuracy of CoGNN remain stable and close to the PlainGNN accuracy as the number of parties/inter-edges increases for all datasets. In contrast, the overall accuracy of FedGNN fluctuates and is consistently lower than CoGNN and PlainGNN. Specifically, from 2-party to 5-party, the accuracy (border accuracy) gap between CoGNN and FedGNN on Cora increases from 3.87% to 11.06% (5.36% to 11.52%); (ii) the accuracy gap between CoGNN and FedGNN is more significant in border vertices for all three datasets. The results reveal that in a GNN collaborative setting where the inter-edges are more intense, training on the global graph using CoGNN brings more accuracy elevation over FedGNN. Overall, collaboratively training GNN using CoGNN achieves better model performance. The computing/communication overhead in FedGNN is certainly small since it trains models in plaintext. Yet it lacks a provable privacy guarantee and promising model performance.

8.2.4 Training/Inference Duration Breakdown. Besides accuracy, we provide the duration breakdowns of CoGNN-Opt in the two-party setting in Table 1 and 2 to show the exact overhead in GCN training/inference. The **OEP Pre** column is the offline OEP duration amortized for each training epoch/inference. The **PreScatter** and **PostGather** columns correspond to the optimizations of CoGNN in § 6.6. The **Scatter Comp** is not applicable here since CoGNN-Opt distributes the vector-scalar multiplication operations to **PreScatter** and **PostGather**. The **Total** column sums up all offline and online durations. The message passing (summing up **OEP Pre**, **Scatter OEP**, **Scatter OGA**, **Gather**) consumes only 2 ~ 6% (2 ~ 5%) of the total training (inference) duration, indicating that our message passing paradigm and OEP/OGA protocols are highly efficient. The heavyweight parts of CoGNN are in **PreScatter**, **PostGather** and **Apply**, because they perform expensive vector-scalar multiplication and NN computations (linear and non-linear). See our technical report for the detailed communication overhead breakdown.

9 FUTURE WORK

Other Types of GNN Models. As a generic framework for collaborative GNN learning, CoGNN provides a high-level abstraction for secure, efficient and scalable GNN training/inference. Nevertheless, instantiating specific GNN models under CoGNN requires dedicated construction of all Dispatch tasks and Collect tasks to express specialized forward/backward computation. Further optimizations like our optimizations for GCN might also be needed to bring out concrete efficiency for computing specific GNN models. We leave the support of other types of GNN models, such as GraphSAGE [19] and graph attention network (GAT) [47], to future work.

Mini-Batch Training. CoGNN is currently designed for full-batch training across the whole global graph. In the real world, there is a need for mini-batch training to support GNN learning on a large graph dataset. Supporting mini-batch training in CoGNN requires an additional design of collaborative mini-batch sampling without violating the privacy of each graph data owner. The sampling process selects out the mini-batch-related graph data, which can be then fed to the CoGNN training/inference process. This is also a promising future direction for extending CoGNN.

Other Scenarios of Graph Collaboration. The graph collaboration setting that CoGNN deals with, i.e., exclusively different vertex sets with inter-edges, is one of the most popular settings of graph data collaboration. However, there are still other scenarios. For example, one party holds vertex features and labels, while another party has the graph topology [20, 54]. They want to put the features/labels and graph topology together to jointly train a GNN model. Such GNN collaboration can be handled by CoGNN

by having the two parties secretly share the features and perform GNN training thereafter. A more complex scenario might be that multiple graph owners have overlapped vertex sets and different graph topologies. The vertex features that they hold are also different from each other. Adapting CoGNN to these scenarios to support efficient GNN collaboration is also included in our future work.

Other Types of Gradient Aggregation Schemes. The way how CoGNN-GCN aggregates the gradients of different subgraphs (in Collect-2) is equivalent to the FedAvg [9] scheme. Thus, in accuracy evaluation, we compare CoGNN with the FedAvg-based FL approach. Supporting other aggregation schemes, like FedOpt [6] and FedProx [29], is part of the future extension of CoGNN.

Other Security Models. CoGNN currently considers semi-honest and non-colluding adversaries. We have explained the rationale behind this security model in § 4. We further discuss the possibility of extending CoGNN to other stronger security models and the extra overhead that might be introduced thereafter. In the n -party setting (i.e., an arbitrary number of parties), to extend CoGNN to malicious security, the main difficulty is in protecting the delegated Collect phase. This goal is similar to verifiable secure aggregation in FL, and can borrow the techniques from recent works (like [18, 44]). This increases computation/communication costs but does not necessarily sacrifice scalability. For colluding security, we need to combine CoGNN with an MPC scheme over t -out-of- n secret sharing. This would sacrifice scalability since the same task has to be processed by multiple parties.

In the setting of a fixed number of parties, like prior SML-based approaches, extending CoGNN to a stronger security model is relatively lightweight. For example, for the 2-party setting (like in [49]), CoGNN requires no share delegation/redistribution, and realizing malicious security mainly needs to protect OEP and OGA. For the 3-party or 4-party setting, we can incorporate the CoGNN message-passing mechanism into a maliciously secure 3PC or 4PC scheme (similar to [5, 26]), but the scalability is degraded.

10 CONCLUSION

This paper presents CoGNN, a secure, efficient and scalable framework for collaborative GNN learning among multiple graph owners. CoGNN introduces a new oblivious message passing paradigm that halves the communication required by the SOTA approach. Based on it, we build a two-stage Dispatch-Collect execution scheme to securely and efficiently express GNN training and inference as distributed vertex-centric computation. By instantiating GCN under the CoGNN framework and performing comprehensive comparisons with prior approaches, we show that CoGNN reduces up to 123x running time and up to 522x communication cost per party when compared to the SML-based SOTA approach, while achieving good scalability as the number of graph owners increases. Meanwhile, the GCN models trained under CoGNN have performance comparable with plaintext global-graph training, and yield up to 11.06% accuracy improvement over GCN models trained via federated learning.

11 ACKNOWLEDGEMENTS

We thank all the anonymous reviewers and our shepherd for their insightful and constructive comments on this paper. The research

is supported in part by NSFC under Grant 62425201 and Grant 62132011.

REFERENCES

- [1] 2020. Complete guide to GDPR compliance. <https://gdpr.eu/> Accessed: 2022-12-08.
- [2] 2023. libOTe. A fast, portable, and easy to use Oblivious Transfer Library. <https://github.com/osu-crypto/libOTe>
- [3] 2023. Secure and Correct Inference (SCI) Library. <https://github.com/mpc-msri/EzPC/tree/master/SCI>
- [4] 2023. troy := seal-cuda. GPU implementation of BFV, CKKS and BGV homomorphic encryption schemes. <https://github.com/lightbulb128/troy>
- [5] Toshinori Araki, Jun Furukawa, Kazuma Ohara, Benny Pinkas, Hanan Rosemarin, and Hikaru Tsuchida. 2021. Secure Graph Analysis at Scale. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security* (Virtual Event, Republic of Korea) (CCS '21). Association for Computing Machinery, New York, NY, USA, 610–629.
- [6] Muhammad Asad, Ahmed Moustafa, and Takayuki Ito. 2020. FedOpt: Towards communication efficiency and privacy preservation in federated learning. *Applied Sciences* 10, 8 (2020), 2864.
- [7] Nuttapong Attrapadung, Hiraku Morita, Kazuma Ohara, Jacob C. N. Schuldt, Tadanori Teruya, and Kazunari Tozawa. 2022. Secure Parallel Computation on Privately Partitioned Data and Applications. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security* (Los Angeles, CA, USA) (CCS '22). Association for Computing Machinery, New York, NY, USA, 151–164.
- [8] Marina Blanton, Ahreum Kang, and Chen Yuan. 2020. Improved Building Blocks for Secure Multi-party Computation Based on Secret Sharing with Honest Majority. In *Applied Cryptography and Network Security*, Mauro Conti, Jianying Zhou, Emiliano Casalicchio, and Angelo Spognardi (Eds.). Springer International Publishing, Cham, 377–397.
- [9] Keith Bonawitz, Vladimir Ivanov, Ben Kreuter, Antonio Marcedone, H Brendan McMahan, Sarvar Patel, Daniel Ramage, Aaron Segal, and Karn Seth. 2017. Practical secure aggregation for privacy-preserving machine learning. In *proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security* (CCS '17). 1175–1191.
- [10] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, Peter Rindal, and Peter Scholl. 2019. Efficient Two-Round OT Extension and Silent Non-Interactive Secure Computation. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security* (London, United Kingdom) (CCS '19). Association for Computing Machinery, New York, NY, USA, 291–308.
- [11] Nishanth Chandran, Divya Gupta, Aseem Rastogi, Rahul Sharma, and Shradha Tripathi. 2019. EzPC: Programmable and Efficient Secure Two-Party Computation for Machine Learning. In *2019 IEEE European Symposium on Security and Privacy (EuroS&P)*. 496–511.
- [12] Melissa Chase, Esha Ghosh, and Oxana Poburinnaya. 2020. Secret-Shared Shuffle. In *Advances in Cryptology – ASIACRYPT 2020*, Shiho Moriai and Huaxiong Wang (Eds.). Springer International Publishing, Cham, 342–372.
- [13] Fahao Chen, Peng Li, Toshiaki Miyazaki, and Celimuge Wu. 2021. Fedgraph: Federated graph learning with intelligent sampling. *IEEE Transactions on Parallel and Distributed Systems* 33, 8 (2021), 1775–1786.
- [14] Daniel Demmler, Thomas Schneider, and Michael Zohner. 2015. ABY-A framework for efficient mixed-protocol secure two-party computation. In *Network and Distributed System Security (NDSS) Symposium*.
- [15] Jiarui Feng, Yixin Chen, Fuhai Li, Anindya Sarkar, and Muhan Zhang. 2022. How powerful are k-hop message passing graph neural networks. *Advances in Neural Information Processing Systems* 35 (2022), 4776–4790.
- [16] C. Lee Giles, Kurt D. Bollacker, and Steve Lawrence. 1998. CiteSeer: An Automatic Citation Indexing System. In *Proceedings of the Third ACM Conference on Digital Libraries* (Pittsburgh, Pennsylvania, USA) (DL '98). Association for Computing Machinery, New York, NY, USA, 89–98.
- [17] Xavier Glorot and Yoshua Bengio. 2010. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics (Proceedings of Machine Learning Research, Vol. 9)*, Yee Whye Teh and Mike Titterton (Eds.). PMLR, Chia Laguna Resort, Sardinia, Italy, 249–256. <https://proceedings.mlr.press/v9/glorot10a.html>
- [18] Changhee Hahn, Hodong Kim, Minjae Kim, and Junbeom Hur. 2023. VerSA: Verifiable Secure Aggregation for Cross-Device Federated Learning. *IEEE Transactions on Dependable and Secure Computing* 20, 1 (2023), 36–52.
- [19] William L. Hamilton, Rex Ying, and Jure Leskovec. 2017. Inductive Representation Learning on Large Graphs (NIPS'17). Curran Associates Inc., Red Hook, NY, USA, 1025–1035.
- [20] Xinlei He, Jinyuan Jia, Michael Backes, Neil Zhenqiang Gong, and Yang Zhang. 2021. Stealing Links from Graph Neural Networks. In *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, 2669–2686.
- [21] Jinzhang Hu, Ruimin Hu, Zheng Wang, Dengshi Li, Junhang Wu, Lingfei Ren, Yilong Zang, Zijun Huang, and Mei Wang. 2023. Collaborative Fraud Detection:

- How Collaboration Impacts Fraud Detection. In *Proceedings of the 31st ACM International Conference on Multimedia (MM '23)*. Association for Computing Machinery, New York, NY, USA, 8891–8899.
- [22] Dejun Jiang, Zhenxing Wu, Chang-Yu Hsieh, Guangyong Chen, Ben Liao, Zhe Wang, Chao Shen, Dongsheng Cao, Jian Wu, and Tingjun Hou. 2021. Could graph neural networks learn better molecular representation for drug discovery? A comparison study of descriptor-based and graph-based models. *Journal of cheminformatics* 13, 1 (2021), 1–23.
- [23] Lindell Katz. 2007. 11.3 *The Paillier Encryption Scheme*, in *Introduction to modern cryptography*. CRC Press, 385–394 pages.
- [24] Mustakimur Rahman Khandaker, Yueqiang Cheng, Zhi Wang, and Tao Wei. 2020. COIN Attacks: On Insecurity of Enclave Untrusted Interfaces in SGX (ASPLOS '20). Association for Computing Machinery, New York, NY, USA, 971–985.
- [25] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *The International Conference on Learning Representations (ICLR Poster)*.
- [26] Nishat Koti, Varsha Bhat Kukkala, Arpita Patra, and Bhavish Raj Gopal. 2023. Entrada to Secure Graph Convolutional Networks. Cryptology ePrint Archive, Paper 2023/1807. <https://eprint.iacr.org/2023/1807>
- [27] Nishat Koti, Arpita Patra, Rahul Rachuri, and Ajith Suresh. 2022. Tetrad: Actively secure 4PC for secure training and inference. In *Network and Distributed System Security (NDSS) Symposium*.
- [28] Qi Li, Zhuotao Liu, Qi Li, and Ke Xu. 2023. martFL: Enabling Utility-Driven Data Marketplace with a Robust and Verifiable Federated Learning Architecture (CCS '23). Association for Computing Machinery, New York, NY, USA.
- [29] Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, and Virginia Smith. 2020. Federated optimization in heterogeneous networks. *Proceedings of Machine learning and systems* 2 (2020), 429–450.
- [30] Xuanqi Liu, Zhuotao Liu, Qi Li, Ke Xu, and Mingwei Xu. 2024. Pencil: Private and Extensible Collaborative Learning without the Non-Colluding Assumption. In *Network and Distributed System Security (NDSS) Symposium*.
- [31] Yanjun Liu, Hongwei Li, Xinyuan Qian, and Meng Hao. 2023. ESA-FedGNN: Efficient secure aggregation for federated graph neural networks. *Peer-to-peer Networking and Applications* 16, 2 (2023), 1257–1269.
- [32] Yi Lu, James Cheng, Da Yan, and Huanhuan Wu. 2014. Large-Scale Distributed Graph Computing Systems: An Experimental Evaluation. *Proc. VLDB Endow.* 8, 3 (nov 2014), 281–292.
- [33] Grzegorz Malewicz, Matthew H. Austern, Aart J.C Bik, James C. Dehnert, Ilan Horn, Naty Leiser, and Grzegorz Czajkowski. 2010. Pregel: A System for Large-Scale Graph Processing. In *Proceedings of the 2010 ACM SIGMOD International Conference on Management of Data (Indianapolis, Indiana, USA) (SIGMOD '10)*. Association for Computing Machinery, New York, NY, USA, 135–146.
- [34] Andrew McCallum, Kamal Nigam, Jason Rennie, and Kristie Seymore. 2000. Automating the Construction of Internet Portals with Machine Learning. *Information Retrieval* 3, 2 (2000), 127–163. <http://dblp.uni-trier.de/db/journals/ir/ir3.html#McCallumNRS00>
- [35] Luca Melis, Congzheng Song, Emiliano De Cristofaro, and Vitaly Shmatikov. 2019. Exploiting unintended feature leakage in collaborative learning. In *2019 IEEE symposium on security and privacy (SP)*. IEEE, 691–706.
- [36] Payman Mohassel and Peter Rindal. 2018. ABY3: A Mixed Protocol Framework for Machine Learning. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (Toronto, Canada) (CCS '18)*. Association for Computing Machinery, New York, NY, USA, 35–52.
- [37] Payman Mohassel and Saeed Sadeghian. 2013. How to Hide Circuits in MPC an Efficient Framework for Private Function Evaluation. In *Advances in Cryptology – EUROCRYPT 2013*, Thomas Johansson and Phong Q. Nguyen (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 557–574.
- [38] P. Mohassel and Y. Zhang. 2017. SecureML: A System for Scalable Privacy-Preserving Machine Learning. In *2017 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, Los Alamitos, CA, USA, 19–38.
- [39] Kit Murdock, David Oswald, Flavio D. Garcia, Jo Van Bulck, Daniel Gruss, and Frank Piessens. 2020. Plundervolt: Software-based Fault Injection Attacks against Intel SGX. In *2020 IEEE Symposium on Security and Privacy (SP)*. 1466–1482.
- [40] Milad Nasr, Reza Shokri, and Amir Houmansadr. 2019. Comprehensive privacy analysis of deep learning: Passive and active white-box inference attacks against centralized and federated learning. In *2019 IEEE symposium on security and privacy (SP)*. IEEE, 739–753.
- [41] Kartik Nayak, Xiao Shaun Wang, Stratis Ioannidis, Udi Weinsberg, Nina Taft, and Elaine Shi. 2015. GraphSC: Parallel Secure Computation Made Easy. In *2015 IEEE Symposium on Security and Privacy (SP)*. 377–394.
- [42] Yang Pei, Renxin Mao, Yang Liu, Chaoran Chen, Shifeng Xu, Feng Qiang, and Blue Elephant Tech. 2021. Decentralized federated graph neural networks. In *International Workshop on Federated and Transfer Learning for Data Sparsity and Confidentiality in Conjunction with IJCAI*.
- [43] Deevashwer Rathee, Mayank Rathee, Nishant Kumar, Nishanth Chandran, Divya Gupta, Aseem Rastogi, and Rahul Sharma. 2020. CryptFlow2: Practical 2-Party Secure Inference. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security (Virtual Event, USA) (CCS '20)*. Association for Computing Machinery, New York, NY, USA, 325–342.
- [44] M. Rathee, C. Shen, S. Wagh, and R. Popa. 2023. ELSA: Secure Aggregation for Federated Learning with Malicious Actors. In *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, Los Alamitos, CA, USA, 1961–1979.
- [45] Prithviraj Sen, Galileo Namata, Mustafa Bilgic, Lise Getoor, Brian Galligher, and Tina Eliassi-Rad. 2008. Collective Classification in Network Data. *AI Magazine* 29, 3 (Sep. 2008), 93. <https://ojs.aaai.org/aimagazine/index.php/aimagazine/article/view/2157>
- [46] Connie Diaz De Teran. 2023. Collaboration Is Key in the Fight Against Anti-Money Laundering. <https://www.paymentsjournal.com/collaboration-is-key-in-the-fight-against-anti-money-laundering/> Accessed: 2023-09-12.
- [47] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. In *The International Conference on Learning Representations (ICLR)*.
- [48] Clement Vignac, Andreas Loukas, and Pascal Frossard. 2020. Building powerful and equivariant graph neural networks with structural message-passing. *Advances in neural information processing systems* 33 (2020), 14143–14155.
- [49] S. Wang, Y. Zheng, and X. Jia. 2023. SecGNN: Privacy-Preserving Graph Neural Network Training and Inference as a Cloud Service. *IEEE Transactions on Services Computing* 16, 04 (jul 2023), 2923–2938.
- [50] Yanling Wang, Jing Zhang, Shasha Guo, Hongzhi Yin, Cuiping Li, and Hong Chen. 2021. Decoupling Representation Learning and Classification for GNN-Based Anomaly Detection. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '21)*. Association for Computing Machinery, New York, NY, USA, 1239–1248.
- [51] Zhen Wang, Weirui Kuang, Yuexiang Xie, Liuyi Yao, Yaliang Li, Bolin Ding, and Jingren Zhou. 2022. FederatedScope-GNN: Towards a Unified, Comprehensive and Efficient Package for Federated Graph Learning. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (Washington DC, USA) (KDD '22)*. Association for Computing Machinery, New York, NY, USA, 4110–4120.
- [52] Bin Wu, Xinyu Yao, Boyan Zhang, Kuo-Ming Chao, and Yinsheng Li. 2023. SplitGNN: Spectral Graph Neural Network for Fraud Detection against Heterophily. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management (CIKM '23)*. Association for Computing Machinery, New York, NY, USA, 2737–2746.
- [53] Chuhan Wu, Fangzhao Wu, Yang Cao, Yongfeng Huang, and Xing Xie. 2021. FedGNN: Federated graph neural network for privacy-preserving recommendation. *arXiv preprint arXiv:2102.04925* (2021).
- [54] Fan Wu, Yunhui Long, Ce Zhang, and Bo Li. 2022. LINKTELLER: Recovering private edges from graph neural networks via influence analysis. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2005–2024.
- [55] Zhiqiang Xie, Minjie Wang, Zihao Ye, Zheng Zhang, and Rui Fan. 2022. Graphiler: Optimizing graph neural networks with message passing data flow graph. *Proceedings of Machine Learning and Systems* 4 (2022), 515–528.
- [56] Ke Zhang, Carl Yang, Xiaoxiao Li, Lichao Sun, and Siu Ming Yiu. 2021. Sub-graph federated learning with missing neighbor generation. *Advances in Neural Information Processing Systems* 34 (2021), 6671–6682.
- [57] Zehong Zhang, Lifan Chen, Feisheng Zhong, Dingyan Wang, Jiaxin Jiang, Sulin Zhang, Hualiang Jiang, Mingyue Zheng, and Xutong Li. 2022. Graph neural network approaches for drug-target interactions. *Current Opinion in Structural Biology* 73 (2022), 102327.
- [58] Zhenhua Zou, Zhuotao Liu, Jinyong Shan, Qi Li, Ke Xu, and Mingwei Xu. 2024. CoGNN: Towards Secure and Efficient Collaborative Graph Learning. Cryptology ePrint Archive, Paper 2024/987. <https://eprint.iacr.org/2024/987>